

INTENSIVÃO DE JAVASCRIPT

Apostila Completa Aula 2

Guia passo a passo para construir seu próprio site
de E-Commerce a partir do zero!



Parte 1

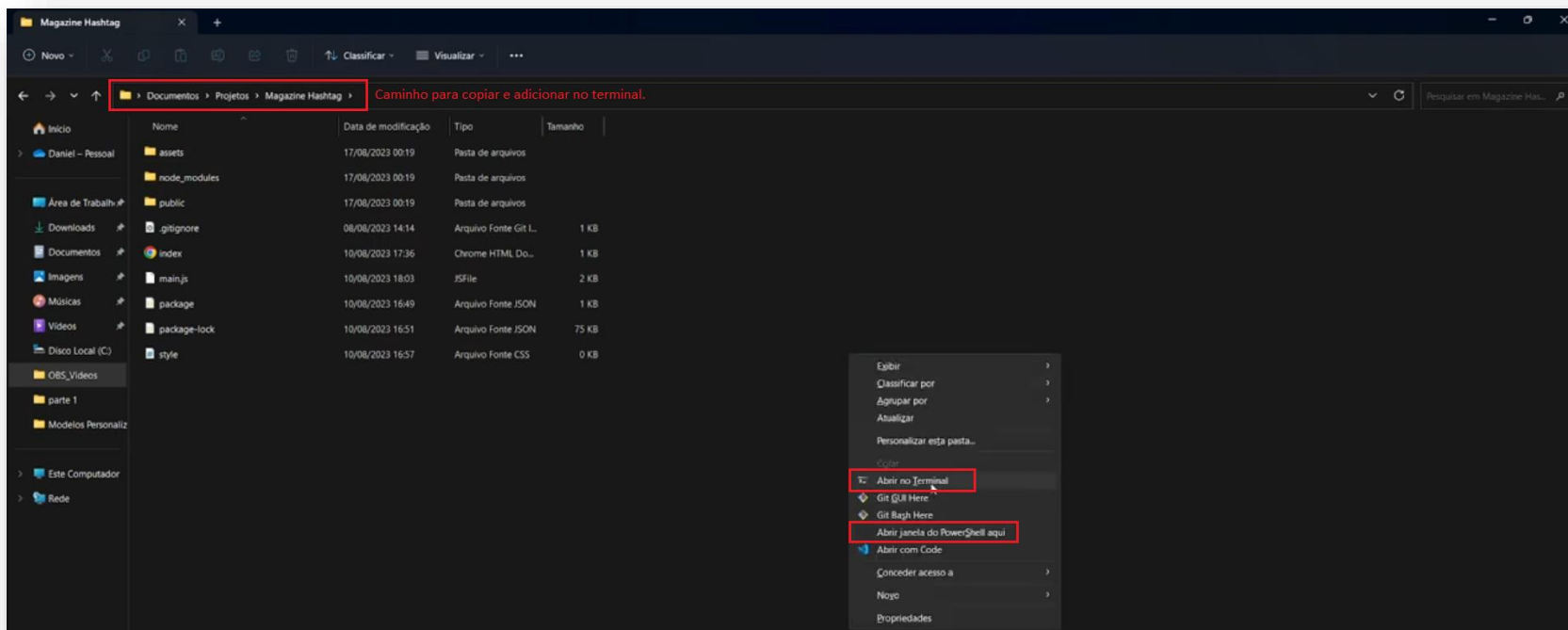
Primeiros passos





Bem-vindo à 2ª aula do nosso Intensivão de Javascript. Vamos dar continuidade ao nosso projeto, nesse momento vamos abrir a pasta que o nosso projeto está localizado, chamada 'Magazine Hashtag' e dentro dela vamos abrir o terminal ou powershell.

Você pode utilizar o botão direito do mouse e clicar em algumas das opções "Abrir no terminal" ou "Abrir o powershell", mas caso não apareça essa opção, aperte a tecla Windows, pesquise "Abrir janela do powershell aqui" e pronto ele terá aberto o nosso terminal.





Para que o nosso terminal olhe para a pasta do projeto, copie o caminho das pastas e digite o comando "cd caminhoPasta" e aperte Enter. O 'cd' significa "change directory" e permite que você mude o diretório atual para um diretório específico.

```
Windows PowerShell
O Windows PowerShell
Copyright (C) Microsoft Corporation. Todos os direitos reservados.

Instale o PowerShell mais recente para obter novos recursos e aprimoramentos! https://aka.ms/PSWindows

PS C:\Users\danie_qf> cd C:\Users\danie_qf\Documents\Projetos\Magazine Hashtag
Set-Location : Não é possível localizar um parâmetro posicional que aceite o argumento 'Hashtag'.
No linha:1 caractere:1
+ cd C:\Users\danie_qf\Documents\Projetos\Magazine Hashtag
+ ~~~~~
+ CategoryInfo          : InvalidArgument: (:) [Set-Location], ParameterBindingException
+ FullyQualifiedErrorId : PositionalParameterNotFound,Microsoft.PowerShell.Commands.SetLocationCommand

PS C:\Users\danie_qf> cd C:\Users\danie_qf\Documents\Projetos\'Magazine Hashtag'
```

Coloque entre aspas simples para não ocorrer o erro acima.

Isso fará com que o nosso terminal esteja dentro da pasta do nosso projeto e consigamos iniciar nossa aplicação Vite com o comando 'npm run dev' e clicar + CTRL no link para acessar o navegador.

```
Windows PowerShell

PS C:\Users\danie_qf\Documents\Projetos\Magazine Hashtag> npm run dev
```

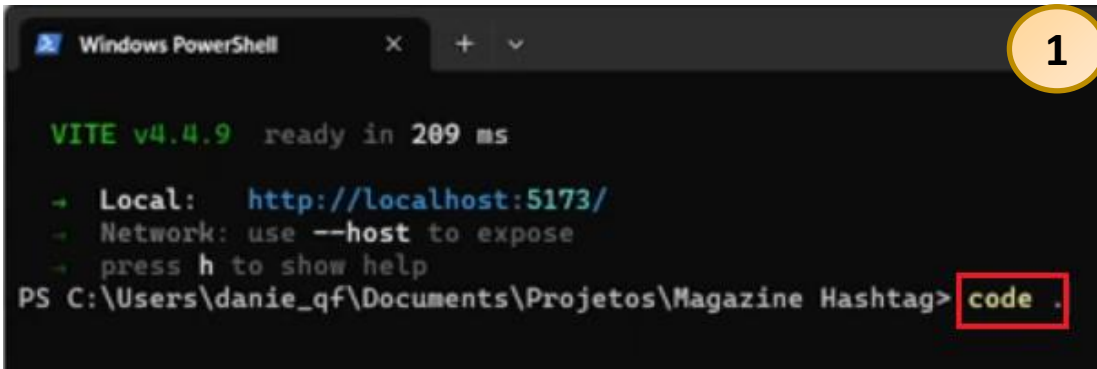
```
> magazine-hashtag@0.0.0 dev
> vite

VITE v4.4.9 ready in 873 ms

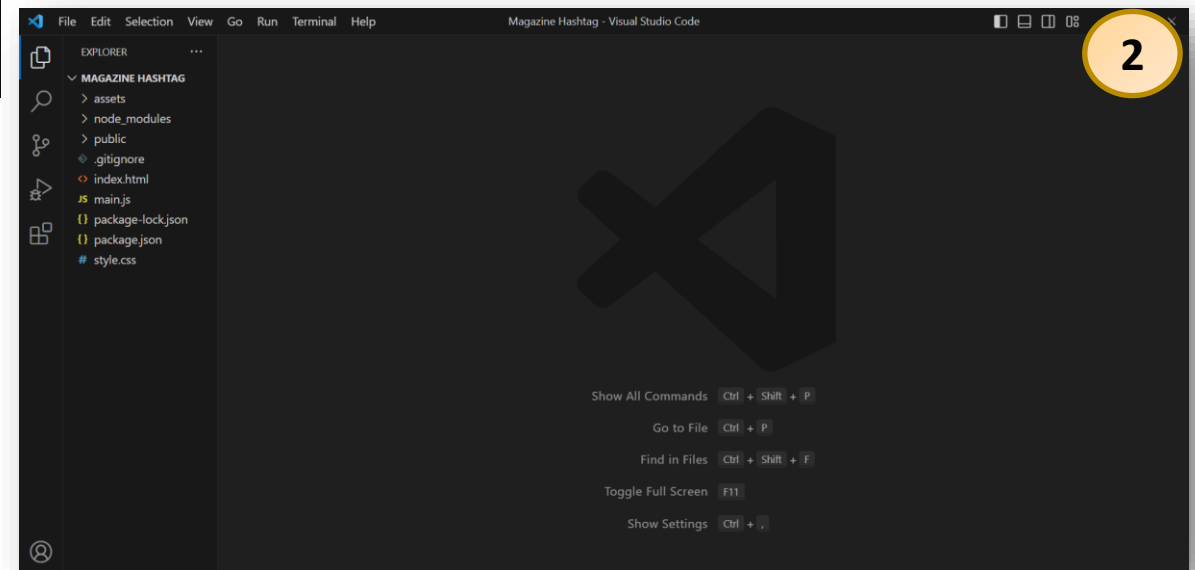
→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h to show help
```



Vamos pausar a nossa aplicação no terminal com **CTRL + C**, para abrir o nosso VS Code via terminal com o comando **code . + Enter** (**Imagem 1**), dessa forma o VS Code será aberto dentro da pasta do nosso projeto, já os arquivos feitos na aula 1 (**Imagem 2**).



```
Windows PowerShell
VITE v4.4.9 ready in 209 ms
  → Local:   http://localhost:5173/
  → Network: use --host to expose
  → press h to show help
PS C:\Users\danie_qf\Documents\Projetos\Magazine Hashtag> code .
```



Parte 2

Instalando Tailwind CSS





O **ID** em CSS é um seletor que permite selecionar um elemento HTML com base em seu atributo ID, que deve ser único e aparecer apenas uma vez no documento.

- Definindo um ID em um elemento HTML:
 - No código HTML, adicione o **atributo id** a um elemento e atribua a ele um valor único.
Por exemplo: `<section id="container-produto">...</section>`
- Selecionando um elemento pelo ID no CSS:
 - Utilize o seletor de ID no CSS para selecionar o elemento desejado. O seletor de ID é representado pelo caractere **'#'** seguido do valor do ID.
Por exemplo: `#container-produto { background-color: aquamarine; }`

```
# style.css  X  <> index.html
# style.css > #container-produto
1  p {
2    color: red;
3  }
4
5  #container-produto {
6    background-color: aquamarine;
7  }
```

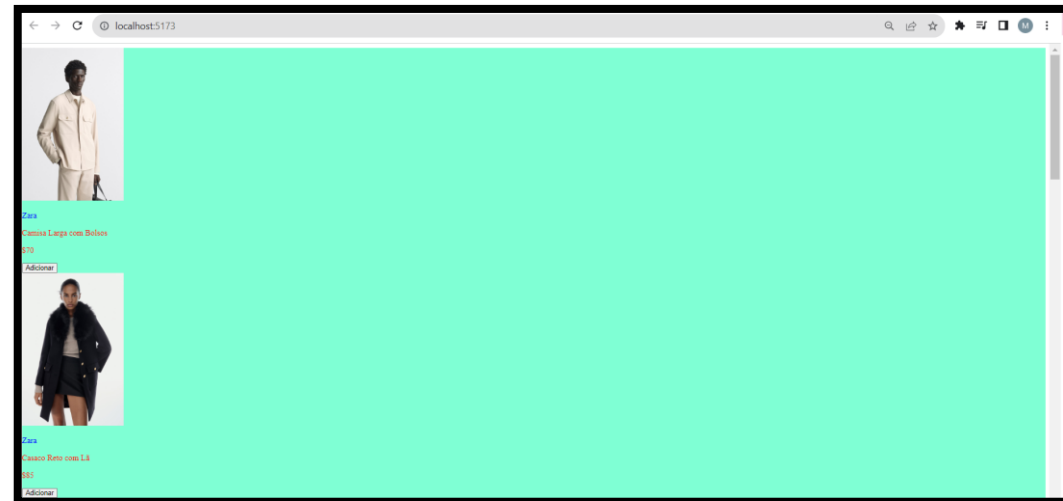




Uma **classe** em CSS é um seletor que permite agrupar elementos HTML com base em um atributo class específico.

- Definindo uma classe em um elemento HTML:
 - No código HTML, adicione o **atributo class** a um elemento e atribua a ele um nome.
Por exemplo: `<p class="marca">...</p>`
- Selecionando elementos por classe no CSS:
 - Utilize o seletor de classe no CSS para selecionar todos os elementos com a classe desejada. O seletor de classe é representado pelo caractere `'.'` seguido do nome da classe.
Por exemplo: `.marca { color: blue; }`

```
# style.css > ...
1  p {
2      color: red;
3  }
4
5  #container-produto {
6      background-color: aquamarine;
7  }
8
9  .marca {
10     color: blue;
11 }
```





O Tailwind CSS é um framework CSS que segue a filosofia "Utility First" e oferece classes utilitárias para criar estilos personalizados de forma eficiente. Ao contrário de outros frameworks CSS, como o Bootstrap, que fornecem componentes pré-estilizados, o Tailwind CSS permite que você crie estilos personalizados combinando classes utilitárias.

A principal vantagem do Tailwind CSS é a flexibilidade que ele oferece. Com as classes utilitárias, você pode criar estilos personalizados sem ter que escrever CSS personalizado para cada elemento. Isso economiza tempo e permite que você personalize seus estilos de forma eficiente.

Ao utilizar o Tailwind CSS, você pode aproveitar as classes agrupadas em grupos de funcionalidades, como cores, tipografia, layout e muito mais. Essas classes são projetadas para serem facilmente compreensíveis e podem ser combinadas para criar estilos complexos.

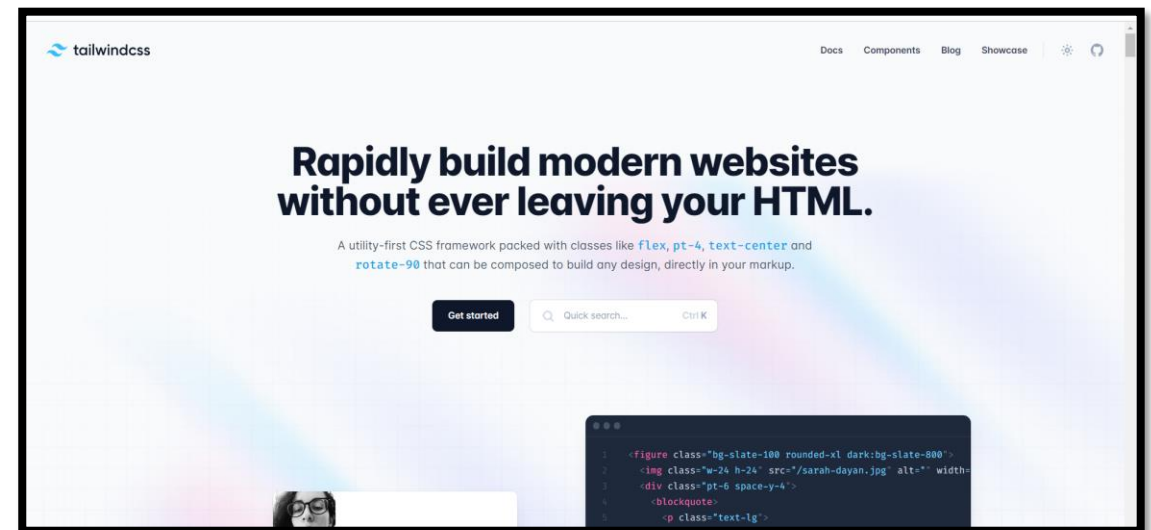
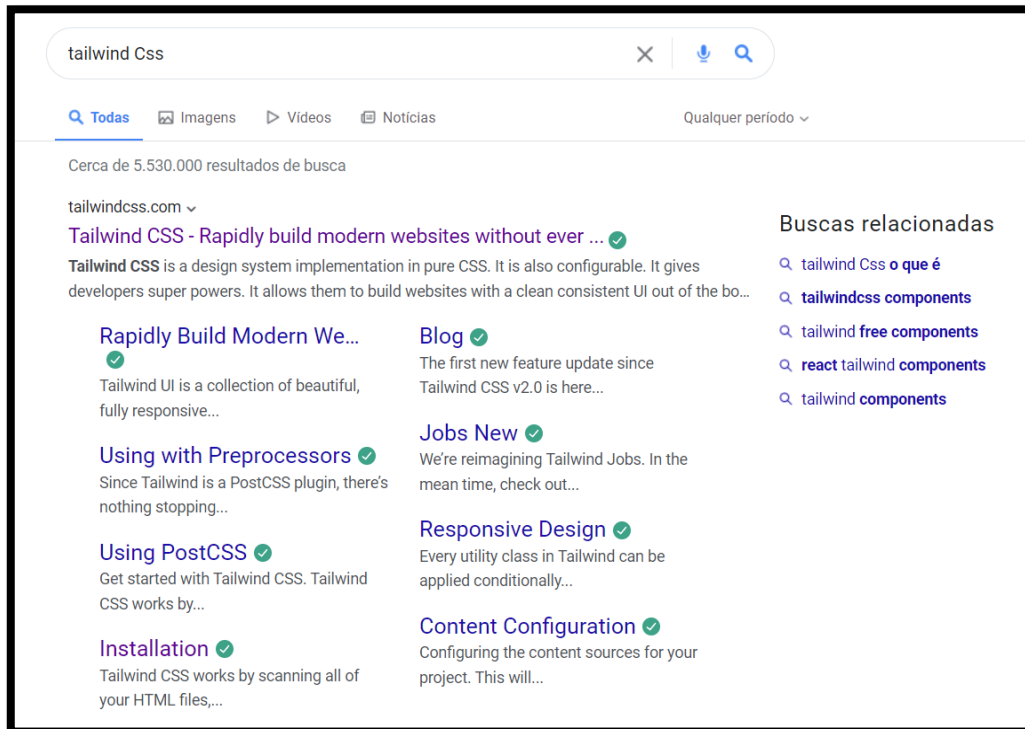
Além disso, o Tailwind CSS possui uma documentação abrangente que fornece exemplos e explicações detalhadas sobre como utilizar as classes utilitárias. Isso facilita o aprendizado e a utilização do framework.

Instalando Tailwind CSS



- Abra o navegador e pesquise no Google por **"tailwindCss"** e clique no primeiro link de retorno da pesquisa. E depois clique em **"Get Started"**.

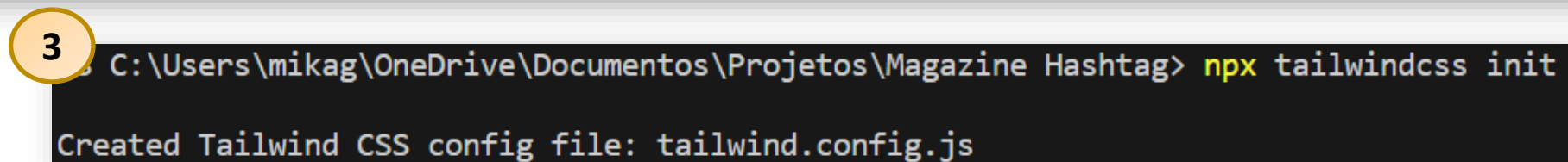
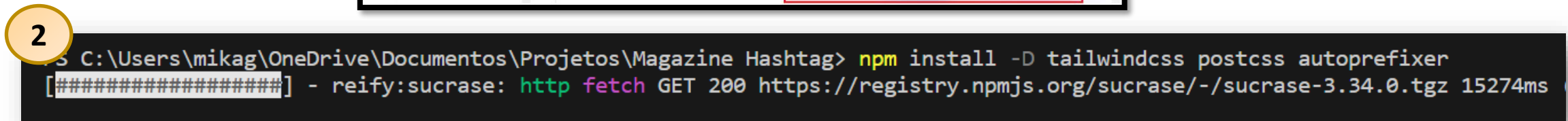
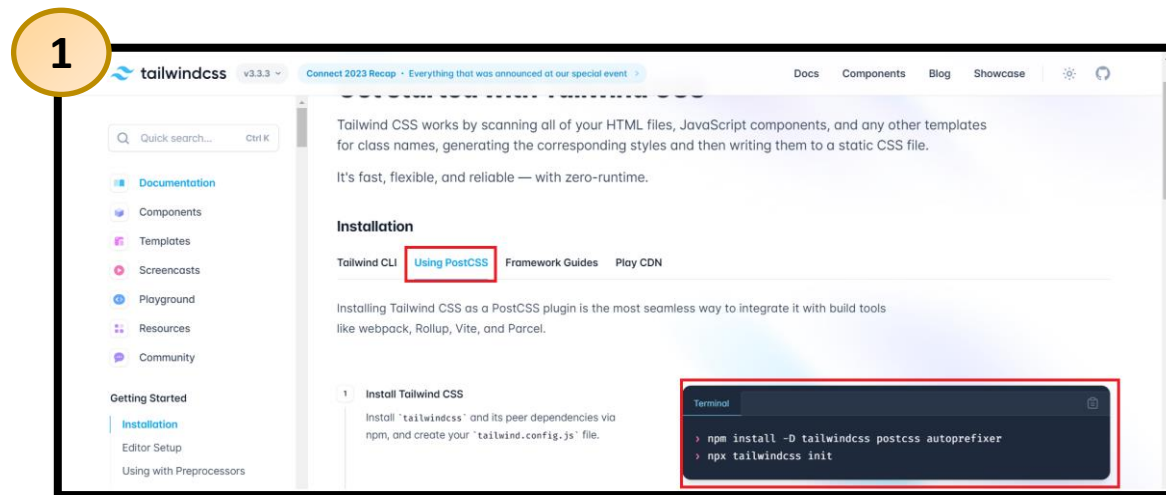
<https://tailwindcss.com/>





Para instalar o Tailwind no nosso projeto, vamos clicar em **Using PostCSS** (Imagem 1):

- Passo 1 - **Install Tailwind CSS**, dentro do terminal copiar os comandos de instalação (Imagem 2 e 3):





Passo 2 - Add Tailwind to your PostCSS configuration:

- Dentro da raiz do nosso projeto vamos criar um arquivo chamado **"postcss.config.js"** e adicionar a seguinte estrutura de código dentro dele:

```
JS postcss.config.js X
JS postcss.config.js > ...
1  export default {
2    plugins: {
3      tailwindcss: {},
4      autoprefixer: {},
5    },
6  };
```

Passo 3 - Configure your template paths:

- Adicione o seguinte código no arquivo **"tailwind.config.js"**, esse arquivo foi criado quando o tailwind foi instalado no projeto:

```
JS tailwind.config.js X
JS tailwind.config.js > ...
1  /** @type {import('tailwindcss').Config} */
2  export default {
3    content: ["./src/**/*.html", "./src/**/*.js"],
4    theme: {
5      extend: {},
6    },
7    plugins: [],
8  }
```



Passo 4 - **Add the Tailwind directives to your CSS:**

- No arquivo `style.css` vamos adicionar as seguintes diretrizes do tailwind:

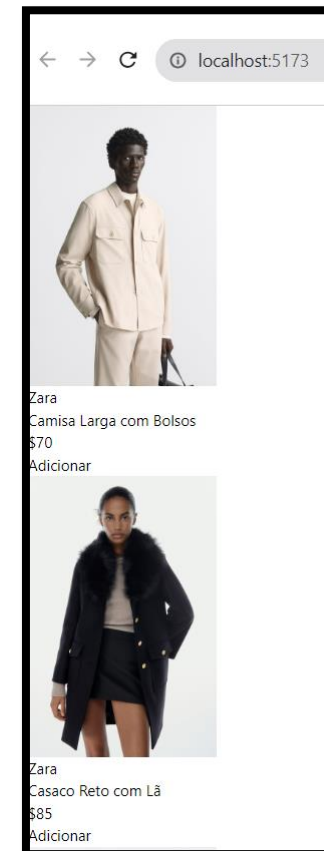
```
# style.css 3 X
```

```
# style.css
```

```
1  @tailwind base;  
2  @tailwind components;  
3  @tailwind utilities;
```

Passo 5 - **Start your build process:**

- Execute novamente o comando no terminal **node run dev**:



Parte 3

Estilização - Card de Produto

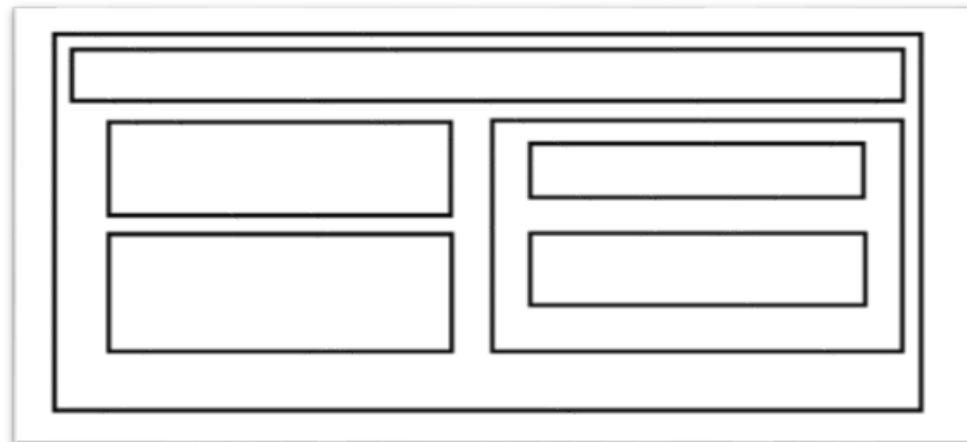


Estilização - Card de Produto

Vamos lembrar de um conceito fundamental para organizar nosso site que é fazer uma comparação entre o HTML e as estruturas retangulares.

O HTML é responsável por definir a estrutura e a organização do conteúdo de um site. Ele permite criar elementos como cabeçalhos, parágrafos, listas, imagens e outros, que compõem a página. Assim como um esqueleto fornece a estrutura básica de um edifício, o HTML fornece a estrutura básica de um site.

Por outro lado, as estruturas retangulares, como divs e sections, são elementos do HTML que podem ser utilizados para criar blocos de conteúdo retangulares em um site. Esses blocos podem ser posicionados, dimensionados e estilizados de diversas maneiras usando o CSS, permitindo a criação de layouts complexos. Podemos comparar as estruturas retangulares a blocos de construção que são usados para criar a aparência visual de um site.



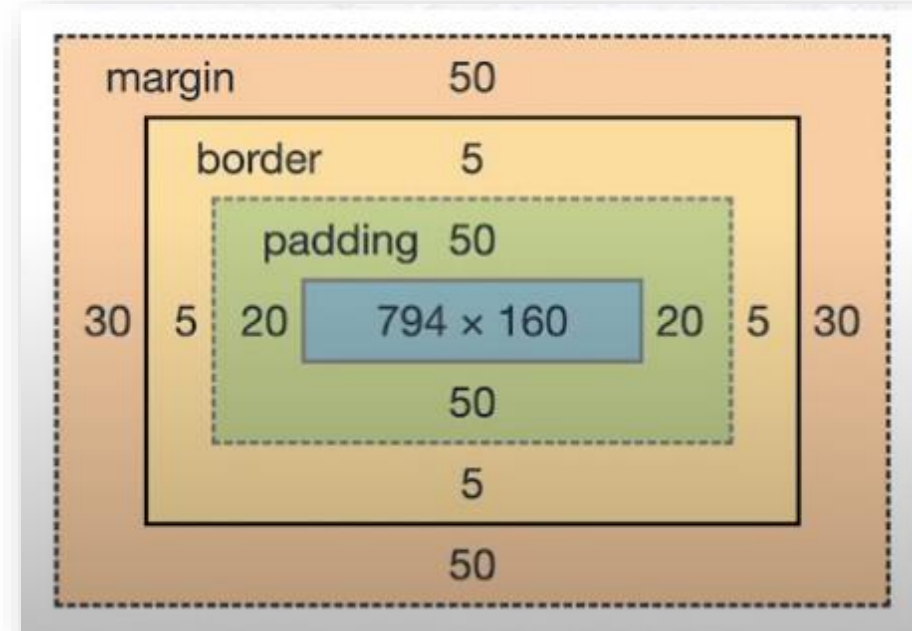


Estilização - Card de Produto

O modelo de caixa (box model) é um conceito fundamental no CSS que define como os elementos HTML são renderizados em uma página. Ele descreve como as diferentes partes de um elemento – conteúdo, padding, borda e margem – se relacionam umas com as outras para formar a aparência final do elemento na página.

Aqui está uma explicação detalhada do modelo de caixa em CSS:

- Cada **elemento HTML** é considerado uma **caixa retangular** com quatro áreas principais: conteúdo, padding, borda e margem.
- O **conteúdo** é a área interna da caixa onde o conteúdo real do elemento é exibido. É controlado pelas propriedades de tamanho, como largura e altura.
- O **padding** é uma área transparente que envolve o conteúdo e separa-o da borda. É controlado pelas propriedades de padding.
- A **borda** é uma linha que envolve o padding e define o limite externo do elemento. É controlada pelas propriedades de borda.
- A **margem** é uma área transparente que envolve a borda e separa o elemento de outros elementos na página. É controlada pelas propriedades de margem.



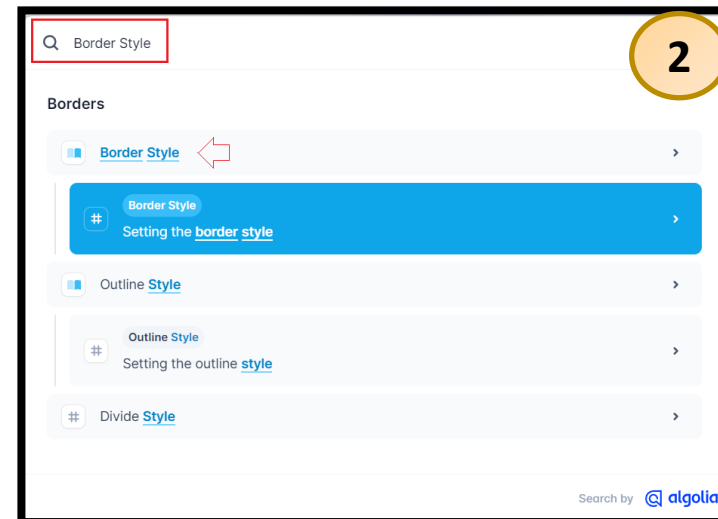
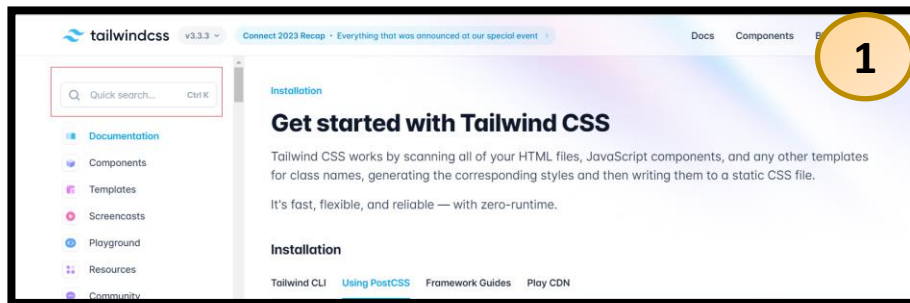


Primeiro vamos adicionar a inteligência ao nosso Card de produtos, para que cada produto tenha seu "id" dinamicamente. No arquivo main.js faça a seguinte alteração no código do nosso Card:

```
69      const cartaoProduto = `<div id="card-produto-${produtoCatalogo.id}">
```

Nesse momento vamos começar a estilizar o nosso Card de produtos, e para isso vamos aplicar uma **classe** do Tailwind para o elemento que está englobando todo ele, que é a nossa **<div>**. A primeira mudança será adicionarmos uma borda ao nosso elemento.

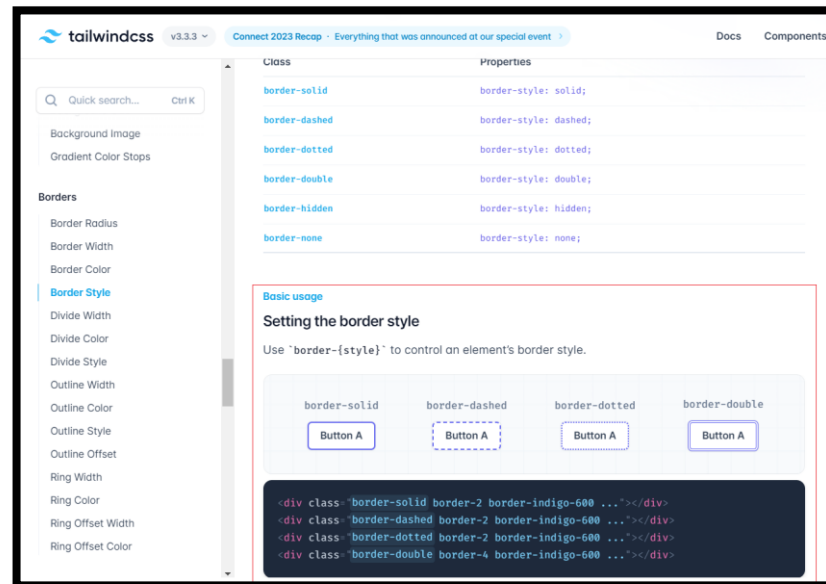
No site oficial do Tailwind, faça uma busca por "Border Style" (Imagem 1 e 2):





Ao utilizar o **Tailwind**, você deve primeiro identificar o elemento que deseja estilizar. O Tailwind, então, oferece uma série de **classes utilitárias** que você pode aplicar a esse elemento. Cada classe utilitária corresponde a uma propriedade CSS específica, o que lhe permite controlar finamente a aparência do elemento.

Além disso, o Tailwind também oferece exemplos de como essas classes utilitárias podem ser aplicadas, permitindo que você visualize como o elemento seria se a classe especificada fosse usada. Para configurar o estilo de borda em Tailwind, você pode usar as classes utilitárias de borda. Por exemplo, para aplicar uma borda sólida de 2 pixels de largura, você pode usar a classe `border-2`.



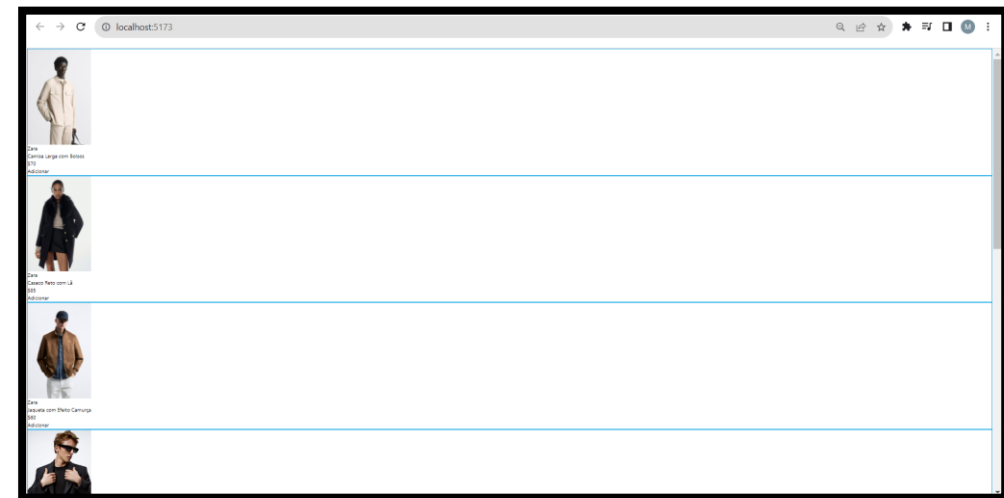


Então, agora que já entendemos um pouco sobre o Tailwind CSS, vamos aplicar algumas de suas classes para criar uma borda em nosso card de produtos.

As classes que vamos utilizar são **border-solid**, **border-2**, e **border-sky-500**.

A classe **border-solid** determina o estilo da borda, neste caso, sólida. Já a classe **border-2** define a largura da borda, que será de 2 pixels. Por fim, a classe **border-sky-500** estabelece a cor da borda, que será um tom de azul (sky 500).

```
JS main.js > [x] cartaoProduto
61   nome: 'Colete Comprido com Cinto',
62   preco: 88,
63   imagem: 'product-8.jpg',
64   feminino: true,
65 },
66 ];
67
68 for (const produtoCatalogo of catalogo) {
69   const cartaoProduto = `<div class="border-solid border-2 border-sky-500"
70   id="card-produto-${produtoCatalogo.id}">
71     
76     <p class="marca">${produtoCatalogo.marca}</p>
77     <p>${produtoCatalogo.nome}</p>
78     <p>${produtoCatalogo.preco}</p>
79     <button>Adicionar</button>
80   </div>`;
81
82   document.getElementById("container-produto").innerHTML += cartaoProduto;
83 }
```

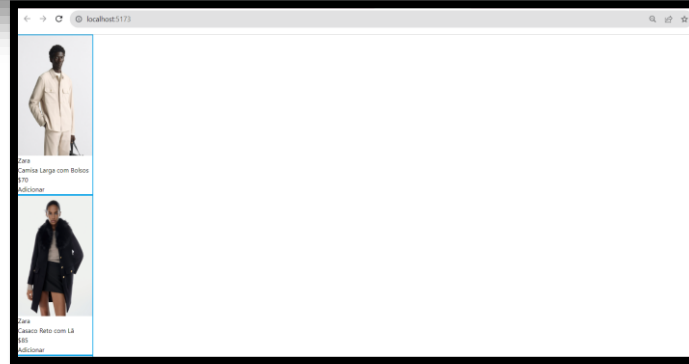




Lembrando que um elemento html pode receber várias classes vamos continuar estilizando o nosso Card, e para isso basta você realizar buscas no Tailwind das propriedades ou elementos que você quer estilizar.

Nesse caso , vamos aplicar no nosso Card uma largura que iremos definir com a classe '**w-48**'.

```
69 | const cartaoProduto = `<div class="border-solid border-2 border-sky-500 w-48">
```



No caso de **w-48**, a largura será fixada em **48 unidades do grid** de espaçamento do Tailwind, que é baseado em um **sistema de espaçamento rem**.

O Tailwind utiliza um sistema de espaçamento padrão que é incrementado em múltiplos de um quarto (0.25) de unidade rem, uma unidade de medida relativa no CSS que é baseada no tamanho de fonte do elemento raiz do documento (geralmente 16px). Então, cada unidade no sistema de espaçamento do Tailwind equivale a 0.25rem. Logo, uma largura de w-48 no Tailwind corresponderá a 12rem ($48 * 0.25\text{rem} = 12\text{rem}$).

Então, quando aplicamos a classe w-48 a um elemento, estamos definindo que a largura desse elemento será de 12rem.



Vamos aplicar no nosso Card uma margem para separar os nossos Cards e iremos utilizar a classe **'my-2'**.

```
69 const cartaoProduto = `<div class="border-solid border-2 border-sky-500 w-48 my-2">
```

A classe **my-2** do Tailwind CSS é usada para definir as **margens verticalmente (topo e base)** de um elemento. No caso de my-2, a margem será de 2 unidades do grid de espaçamento do Tailwind. Então, quando aplicamos a classe my-2 a um elemento, estamos definindo que a margem superior e inferior desse elemento será de 0.5rem.



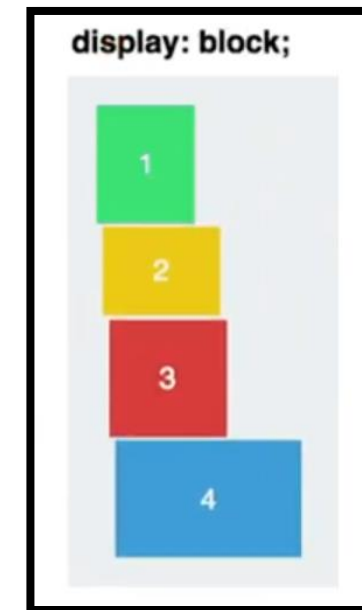
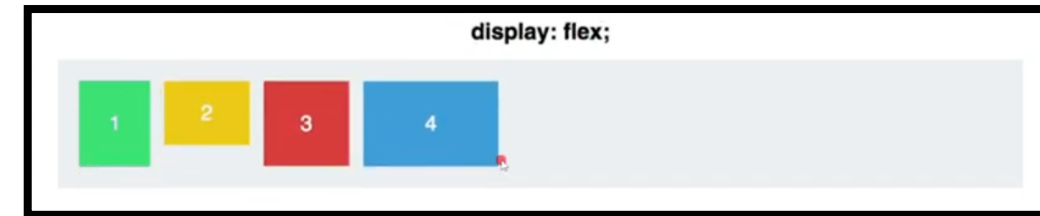


Estilização - Card de Produto

Ao criar páginas na web, podemos imaginar as divs como caixas ou retângulos que contêm o conteúdo dentro delas. Por padrão, essas caixas são empilhadas verticalmente, uma em cima da outra. No entanto, o CSS nos fornece uma ferramenta chamada **Flexbox**, que é representada pela **propriedade display: flex**. Esta ferramenta nos permite alterar a forma como essas caixas são organizadas e como o conteúdo dentro delas é alinhado e distribuído.

Quando aplicamos **display: flex** a uma div, essa div se transforma em um **container flexível**. Isso quer dizer que todos os elementos filhos dessa div (as caixas dentro da caixa maior) se tornam itens flexíveis, e o comportamento deles é alterado. Eles não são mais empilhados verticalmente, mas dispostos em uma linha, lado a lado.

Além disso, ao usar **display: flex**, ganhamos acesso a uma série de outras propriedades que nos permitem controlar como os itens flexíveis são alinhados e distribuídos dentro do **container flexível**. Isso nos dá muito mais controle sobre o layout e facilita a criação de designs responsivos, que se adaptam a diferentes tamanhos de tela.



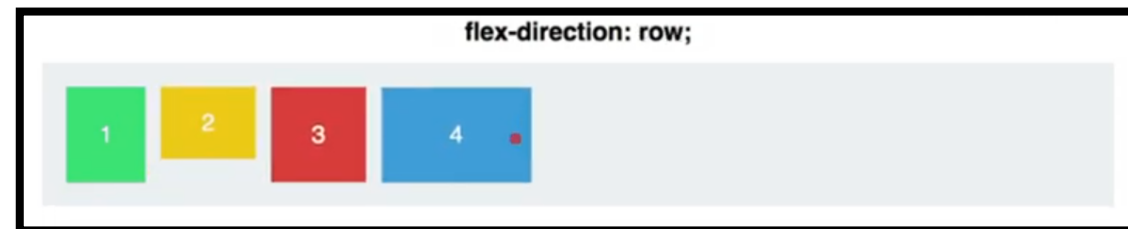
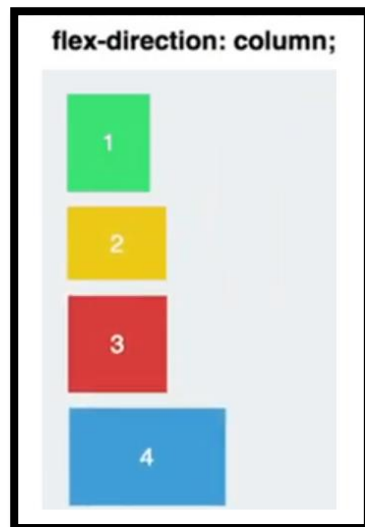


Estilização - Card de Produto

No CSS Flexbox, temos dois eixos principais: o eixo principal e o eixo transversal. O fluxo dos itens flexíveis é definido ao longo desses eixos.

O **eixo principal** é o eixo ao longo do qual os itens flexíveis são dispostos dentro do container. A direção desse eixo é determinada pela **propriedade flex-direction**. Por padrão, o valor desta propriedade é row, o que significa que o eixo principal é o mesmo que o eixo da linha de texto, geralmente da esquerda para a direita.

O **eixo transversal** é perpendicular ao eixo principal e sua direção é determinada pela propriedade flex-direction e pela direção da escrita do texto. Por exemplo, se **flex-direction** é row (a direção padrão), então o eixo transversal corre de cima para baixo.



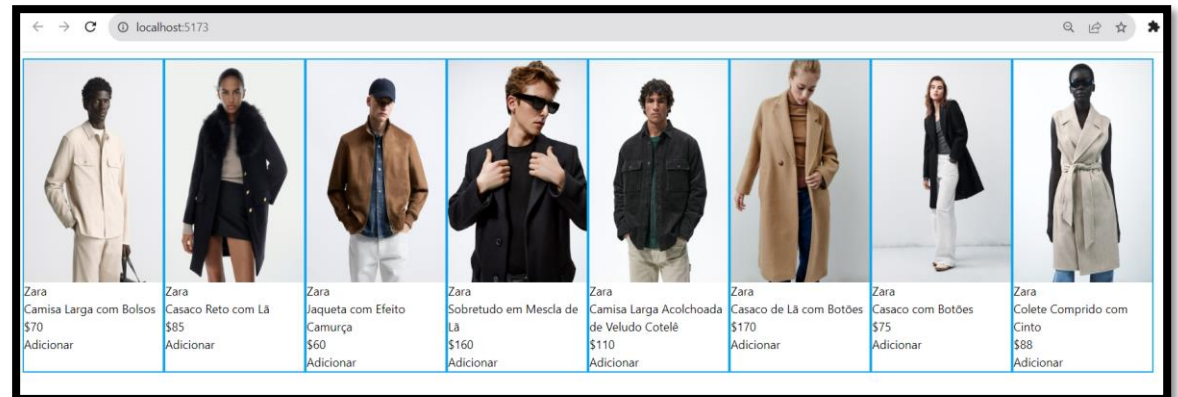


No CSS Flexbox, temos dois eixos principais: o eixo principal e o eixo transversal. O fluxo dos itens flexíveis é definido ao longo desses eixos.

O **eixo principal** é o eixo ao longo do qual os itens flexíveis são dispostos dentro do container. A direção desse eixo é determinada pela **propriedade flex-direction**. Por padrão, o valor desta propriedade é row, o que significa que o eixo principal é o mesmo que o eixo da linha de texto, geralmente da esquerda para a direita.

O **eixo transversal** é perpendicular ao eixo principal e sua direção é determinada pela propriedade flex-direction e pela direção da escrita do texto. Por exemplo, se **flex-direction** é row (a direção padrão), então o eixo transversal corre de cima para baixo.

```
<> index.html > html
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <meta name="viewport" content="width=device-width, initial-scale=1.0">
6    <title>Título da página</title>
7    <link rel="stylesheet" href="style.css">
8  </head>
9  <body>
10   <header></header>
11   <main>                                Contêiner do Card de Produtos
12   <section id="container-produto" class="flex"></section>
13   </main>
14   <script src="main.js"></script>      Classe que transforma em
15   </body>                                um contêiner flexível
16 </html>
```



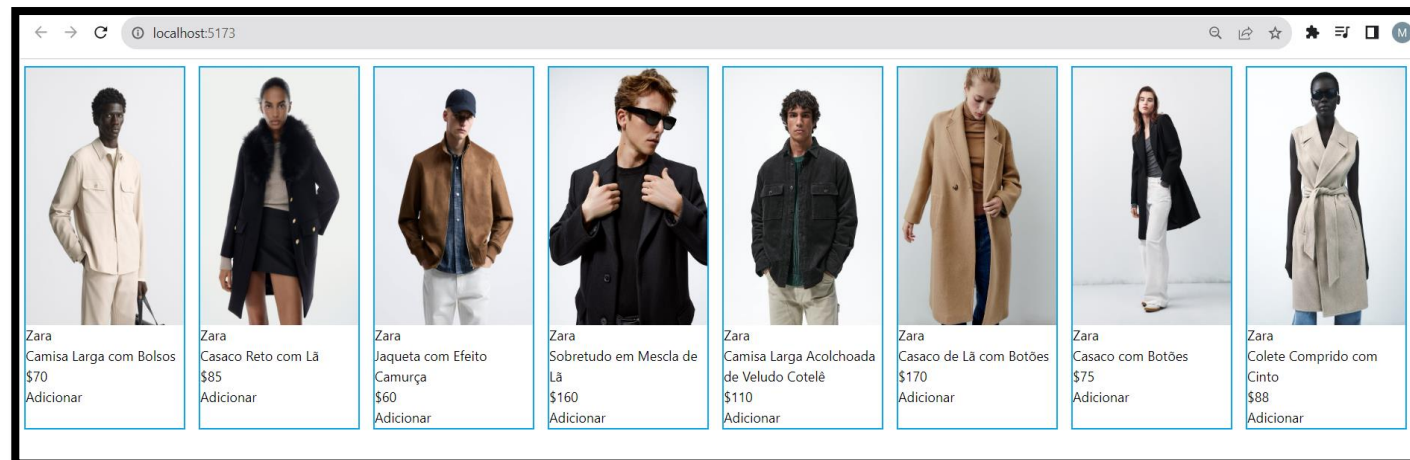


Agora que transformamos nosso container em um Flexbox, precisamos fazer novamente um ajuste na margem e em mais alguma propriedades:

Para redefinir nossa margem, vamos alterar a classe utilitária 'my-2' por 'm-2'. A **classe 'm-2'** no Tailwind CSS é uma classe utilitária que define a margem de um elemento. A letra 'm' representa 'margin' em inglês, que significa margem em português. O número '2' indica o tamanho da margem, nesse caso, é uma margem média.

```
69      const cartaoProduto = `<div class="border-solid border-2 border-sky-500 w-48 m-2">
```

Essas **classes utilitárias no Tailwind CSS** seguem uma **convenção de nomenclatura consistente**. A primeira letra representa a propriedade CSS (no caso, 'm' para margin), seguida por um traço e o valor da propriedade.



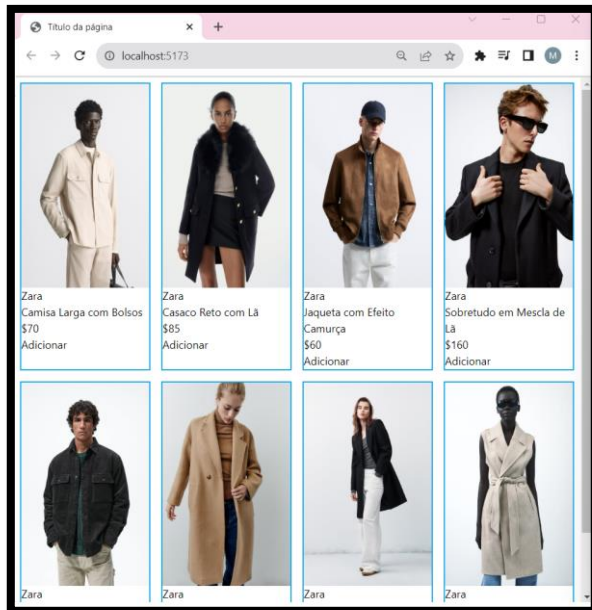


Novamente no nosso container vamos adicionar a classe 'flex-wrap'.

A classe **flex-wrap** é uma classe do Tailwind CSS que controla o comportamento de quebra de linha dos elementos em um container flexível. Ela permite definir se os elementos devem ser exibidos em uma única linha ou se podem ser quebrados em várias linhas. Isso é útil quando você deseja controlar como os elementos são dispostos em um layout flexível.

12

```
<section id="container-produto" class="flex flex-wrap"></section>
```



Nesse momento os nossos Cards de produtos terão essa aparência, e vamos deixar eles assim por enquanto.

Parte 4

Construindo um Header Eficiente

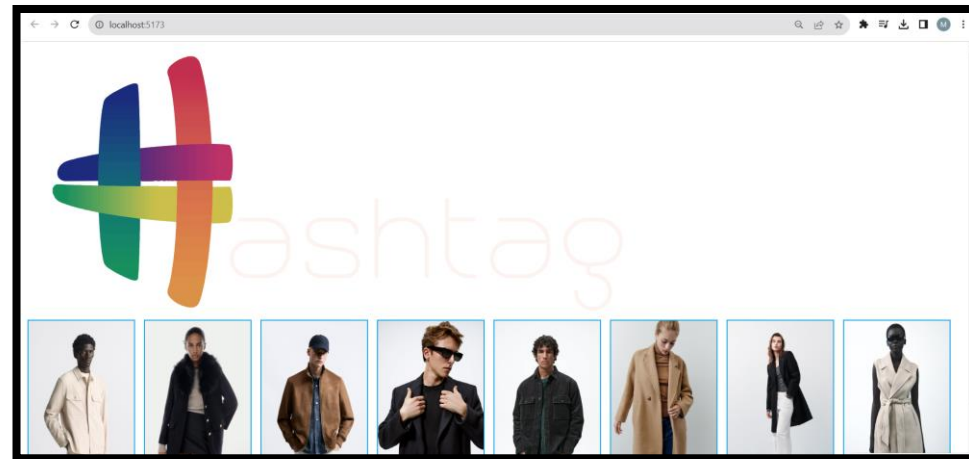


Construindo um Header Eficiente

Nesta parte, vamos explorar os elementos essenciais e as melhores práticas para criar um **Header** impactante e funcional para um site de E-commerce. O Header é a parte superior do site e é geralmente a primeira coisa que os visitantes veem quando acessam uma página. É fundamental que o Header seja bem-estruturado e projetado para proporcionar uma boa experiência de navegação aos usuários.

Primeiro vamos adicionar o logo do nosso site no arquivo index.html, utilizando o elemento `<img>`:

```
<> index.html > html
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <meta name="viewport" content="width=device-width, initial-scale=1.0">
6    <title>Título da página</title>
7    <link rel="stylesheet" href="style.css">
8  </head>
9  <body>
10   <header>
11     
12   </header>
13   <main>
14     <section id="container-produto" class="flex flex-wrap"></section>
15   </main>
16   <script src="main.js"></script>
17 </body>
18 </html>
```



Repare que a proporção do nosso logo está muito grande para o Header (cabeçalho) da nossa página, mas logo resolveremos esse problema.



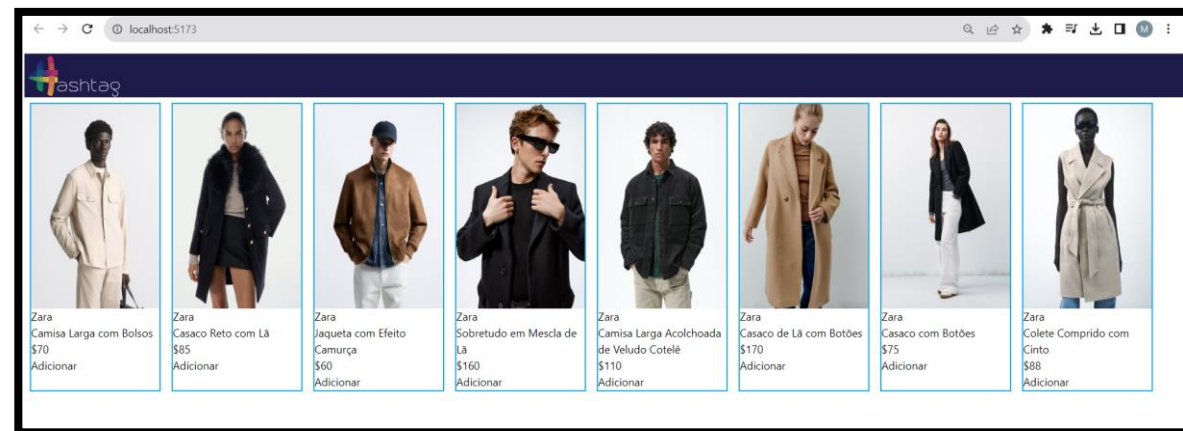
Construindo um Header Eficiente

Vamos iniciar a estilização do nosso Header e para isso vamos utilizar algumas classes do Tailwind buscando criar o layout que desejamos. Utilizaremos a seguintes classes inicialmente:

A classe **flex bg-indigo-950** em Tailwind CSS combina duas classes utilitárias para estilizar elementos HTML. A **classe flex** é utilizada para criar um **container flexível**, permitindo controlar a direção, alinhamento e dimensionamento dos elementos filhos dentro desse container. Por sua vez, a **classe bg-indigo-950** define a **cor de fundo** como um **tom específico de indigo**, criando uma aparência visual personalizada.

E adicionaremos uma altura para a imagem do nosso logo com a classe **'h-16'**, o prefixo "h" indica que essa classe está relacionada à altura (**height**) do elemento. O valor "16" indica que a altura será definida como 16 unidades de medida.

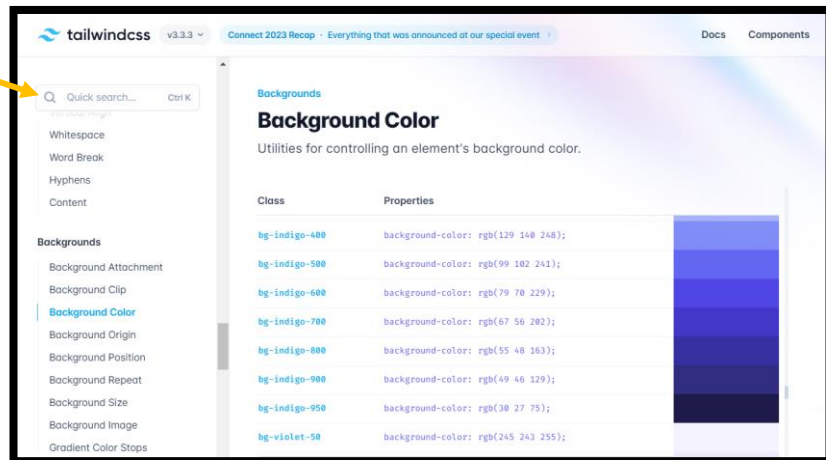
```
index.html > html
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Título da página</title>
7   <link rel="stylesheet" href="style.css">
8 </head>
9 <body>
10   <header class="flex bg-indigo-950">
11     
16   </header>
17   <main>
18     <section id="container-produto" class="flex flex-wrap"></section>
19   </main>
20   <script src="main.js"></script>
21 </body>
22 </html>
```





Construindo um Header Eficiente

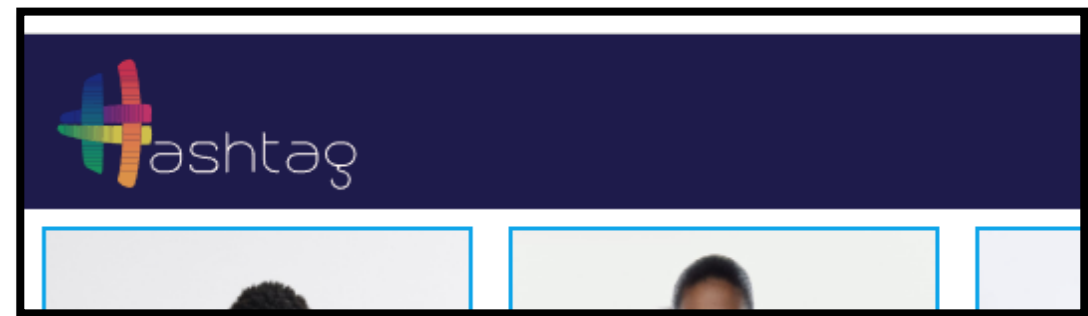
Lembrando que você não precisa saber ou decorar todas as propriedades e estilos que você quer aplicar no seu projeto, o Tailwind possui um **campo de busca** para você procurar e identificar qual estilização você quer aplicar nos seus elementos, e você pode realizar testes adicionando as classes e vendo os resultados no seu navegador.



Continuando a estilização do nosso **Header**. Vamos criar um espaço interno ao redor do conteúdo, ajudando a separá-lo das bordas do elemento utilizando a **propriedade padding**.

Aplice a classe '**p-2**' no seu elemento e ele terá seu padding ajustado em 2 unidades de medida.

```
10 <header class="flex bg-indigo-950 p-2">
```



Exemplo do padding aplicado no Header.

Construindo um Header Eficiente



Adicionaremos dois elementos de botões ao Header, e para isso você pode utilizar o atalho no VS Code:

```
10 <header class="flex bg-indigo-950 p-2">
11   
16   button*2
17 </header>
```

Emmet Abbreviation

E ao apertar **Enter**, dois elementos **button** serão criados:

```
<? index.html > <? html > <? body > <? header.flex.bg-indigo-950.p-2
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Título da página</title>
7   <link rel="stylesheet" href="style.css">
8 </head>
9 <body>
10   <header class="flex bg-indigo-950 p-2">
11     
16     <button></button><button></button>
17   </header>
18   <main>
19     <section id="container-produto" class="flex flex-wrap"></section>
20   </main>
21   <script src="main.js"></script>
22 </body>
23 </html>
```

Dentro de cada **elemento button** adicionaremos os textos '**histórico**' e '**carrinho**' que irão corresponder ao que o botão irá representar no nosso site:

```
10 <header class="flex bg-indigo-950 p-2">
11   
16   <button>histórico</button>
17   <button>carrinho</button>
18 </header>
```

E o nosso Header terá esse formato nesse momento:





Construindo um Header Eficiente

Para organizar os elementos do nosso Header, crie um elemento **<div>** que irá encapsular os dois elementos de botões criados e adicionaremos a **cor branco** para os textos deles.

```
16 <div class="text-slate-200">
17   <button>histórico</button>
18   <button>carrinho</button>
19 </div>
```



A **classe text-slate-200** em Tailwind CSS é uma classe utilitária que define a cor do texto como um tom específico de cinza/branco chamado "slate-200".

No código fornecido, temos uma **<div>** que possui essa classe aplicada. Dentro dessa **<div>**, temos **dois botões: "histórico" e "carrinho"**.

Isso significa que o texto dentro dos botões terá a cor definida pela classe text-slate-200, ou seja, um tom de cinza/branco específico.



Construindo um Header Eficiente

Para posicionar os elementos nos locais que queremos dentro do nosso Header, vamos entender sobre a **propriedade justify-content**, que só poder ser aplicado caso esteja dentro de elemento com a **propriedade flex**, ou seja, que seja um container flexível.

A **propriedade justify-content** em CSS é responsável por definir o alinhamento dos itens em um container flexível ao longo do eixo principal, que é o eixo horizontal por padrão. Essa propriedade permite que você controle a distribuição do espaço entre os itens e seu posicionamento dentro do container.

Existem várias opções de valores que podem ser atribuídos à propriedade justify-content. Vamos explorar algumas delas:

- **flex-start:** Alinha os itens desde o início do container. Os itens são posicionados no lado esquerdo do container, mantendo a ordem original.
- **flex-end:** Alinha os itens até o final do container. Os itens são posicionados no lado direito do container, mantendo a ordem original.
- **center:** Centraliza os itens ao longo do eixo principal do container. Os itens são distribuídos igualmente de forma que o espaço vazio seja dividido por igual antes e depois do primeiro e último item.
- **space-between:** Distribui o espaço igualmente entre os itens. O primeiro item é alinhado no início, o último item é alinhado no final e o espaço restante é dividido igualmente entre os itens.



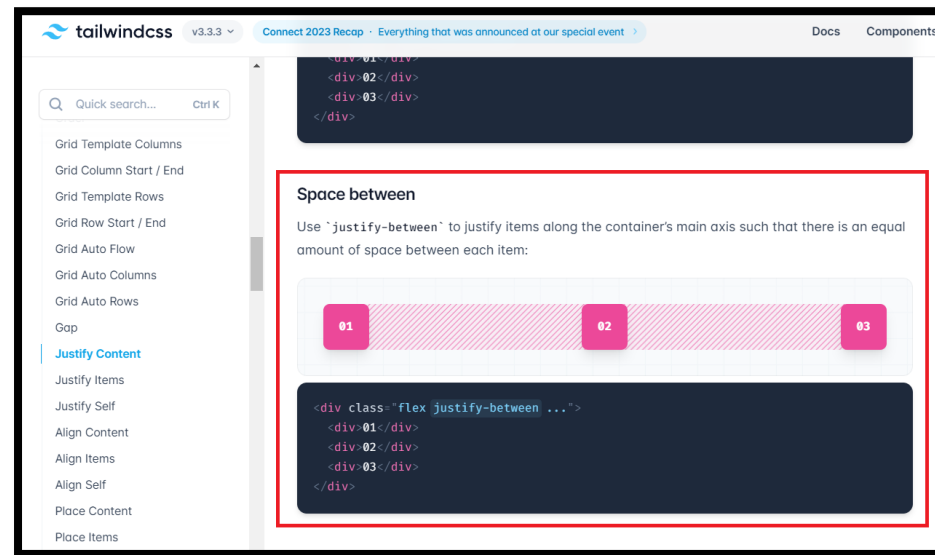
Construindo um Header Eficiente

Para posicionar os elementos nos locais que queremos dentro do nosso Header, vamos entender sobre a **propriedade justify-content**, que só poder ser aplicado caso esteja dentro de elemento com a **propriedade flex**, ou seja, que seja um container flexível.

```
10 | <header class="flex bg-indigo-950 p-2 justify-between">
```



E você pode pesquisar sobre essa propriedade no Tailwind, e ele mostrará todos os tipos de valores que o **justify-content** pode receber, e fornece um exemplo de como os elementos serão posicionados a cada valor.





Construindo um Header Eficiente

Agora vamos iniciar a estilização dos nosso botões, primeiro a **<div>** que engloba esses elemento terá a **classe flex** e vamos adicionar um espaço entre eles com a **classe 'gap-2'**.

A propriedade **'gap-2'** em CSS não é uma propriedade padrão. Ela é uma classe personalizada que fornece um espaçamento consistente entre elementos em um layout.



```
<div class="text-slate-200 flex gap-2">  
  <button>histórico</button>  
  <button>carrinho</button>  
</div>
```



Construindo um Header Eficiente

A última alteração que faremos no posicionamento dos nossos elementos buttons será coloca-los próximo a borda inferior do Header, isso é possível utilizando a classe do Tailwind **'items-end'** que irá aplicar a propriedade **align-items: flex-end**.

A **propriedade align-items** em CSS é utilizada para alinhar os itens de um contêiner flexível ao longo do eixo transversal.

- **flex-end:** alinha os itens ao final do contêiner no eixo transversal.

End

Use `'items-end'` to align items to the end of the container's cross axis:



```
<div class="flex items-end ...">
  <div class="py-4">01</div>
  <div class="py-12">02</div>
  <div class="py-8">03</div>
</div>
```

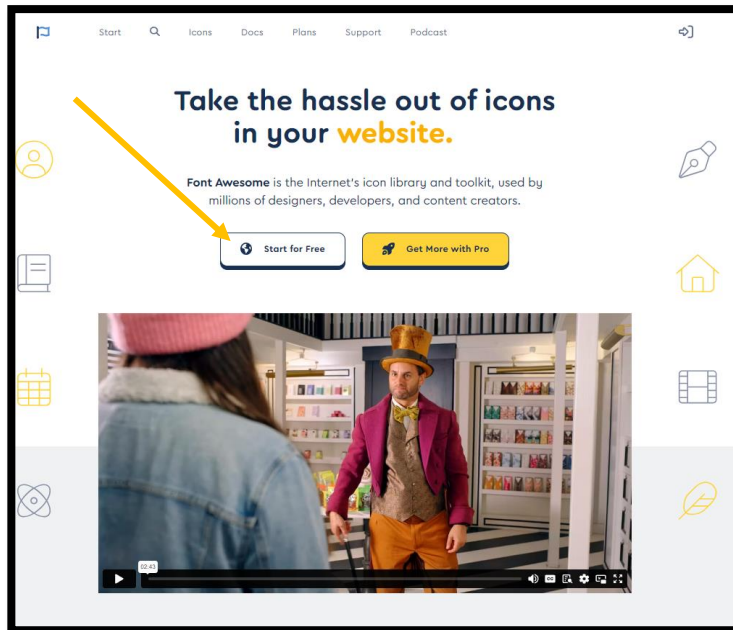
```
16 <div class="text-slate-200 flex gap-2 items-end">
17   <button>histórico</button>
18   <button>carrinho</button>
19 </div>
```



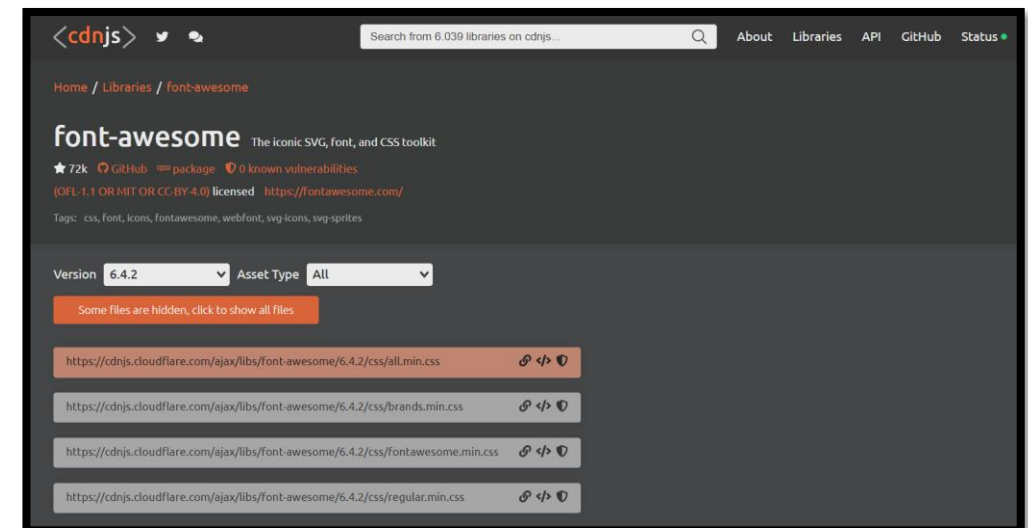
Construindo um Header Eficiente

Para concluir a estilização dos elementos de botão, vamos remover os textos que usamos apenas para posicioná-los no Header e substituí-los por ícones. Para fazer isso, vamos usar uma biblioteca externa chamada "**Font Awesome**". Essa biblioteca nos permite adicionar facilmente ícones aos nossos botões, melhorando o design e a aparência do nosso site.

1 - Vamos pesquisar o site oficial do Font Awesome : <https://fontawesome.com/>



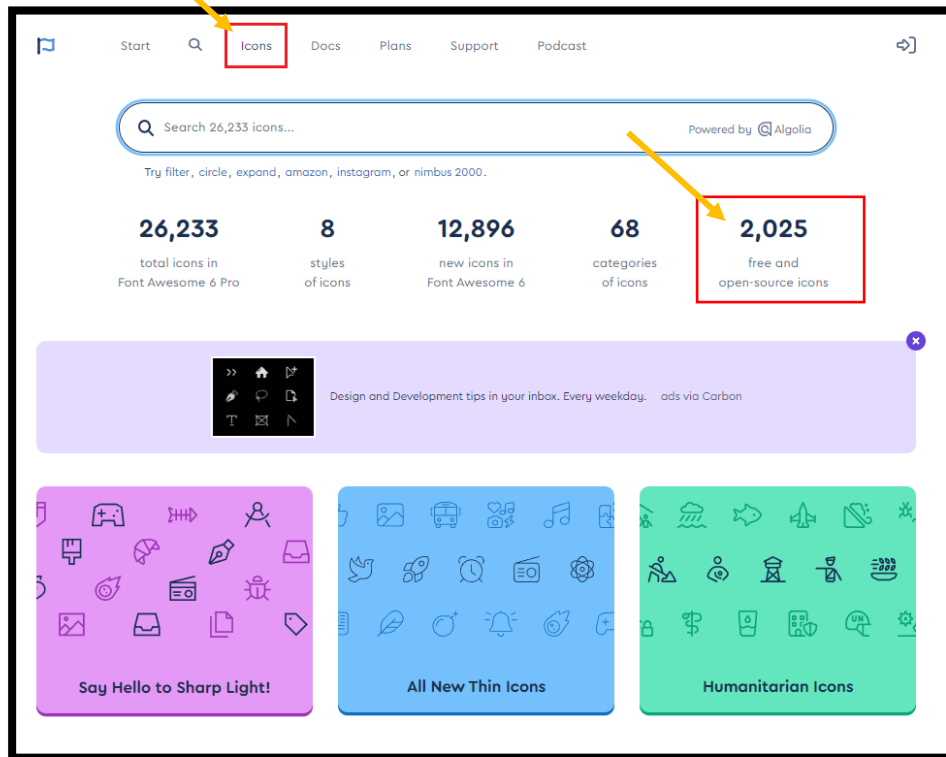
2 - Abrir em uma nova aba o site <https://cdnjs.com/libraries/font-awesome/>



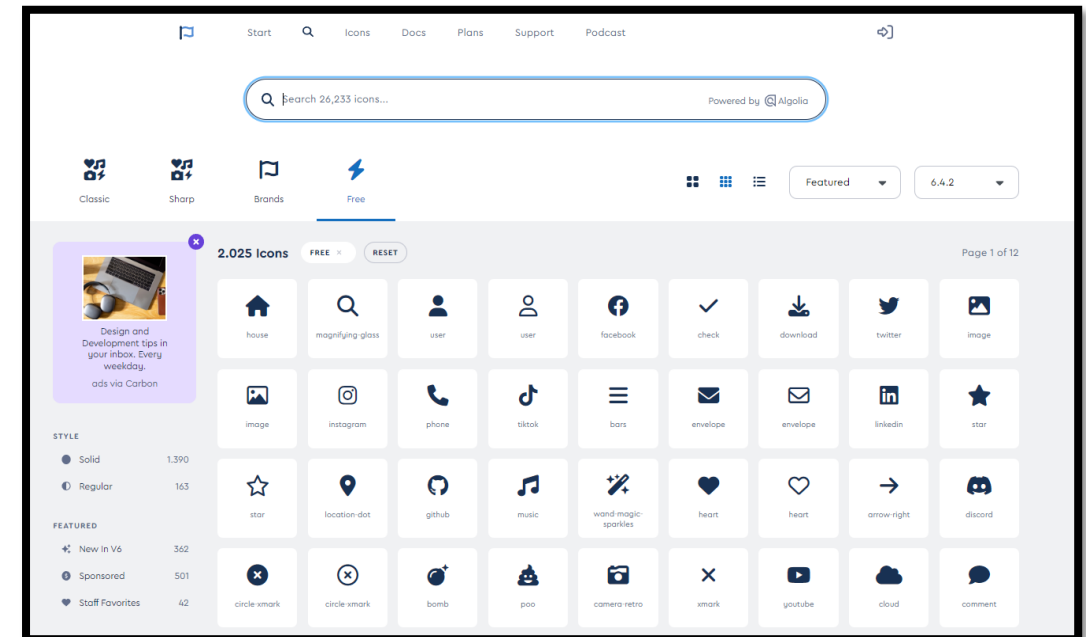
Construindo um Header Eficiente



3 – Clique na aba **'Icons'** do site Font Awesome:



4 – Clique em **'free and open-source icons'** como na imagem ao lado, e ao clicar você será direcionado a página da imagem abaixo:





Construindo um Header Eficiente

Agora que sabemos o local que iremos pesquisar e obter os ícones para os nossos botões, precisamos entender como utilizamos os ícones do site Font Awesome. Vamos navegar agora pelo segundo site que abrimos do CDNJS.

O **CDNJS** é um serviço de CDN (**Content Delivery Network**) que hospeda várias bibliotecas e recursos populares utilizados em desenvolvimento web. Ele fornece uma maneira fácil de incluir essas bibliotecas em seus projetos, reduzindo o tempo de carregamento e melhorando o desempenho do seu site.

5 – Copie o primeiro código clicando no ícone <> (Copy Link Tag):

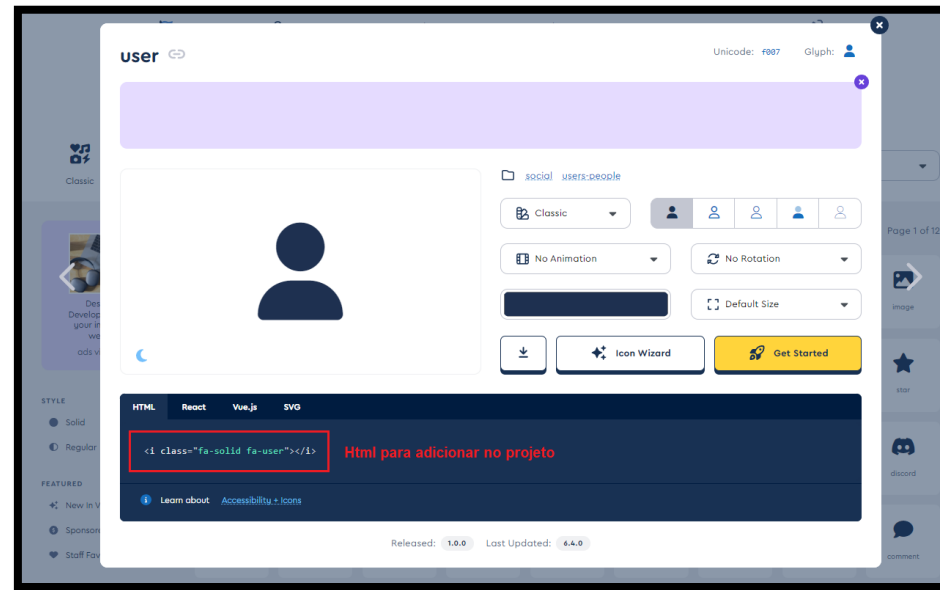




6 – Adicionaremos o link copiado no nosso arquivo index.html, dentro da **tag <head>**:

```
index.html > html > head
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Título da página</title>
7   <link rel="stylesheet" href="style.css">
8   <link
9     rel="stylesheet"
10    href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.2/css/all.min.css"
11    integrity="sha512-z3glpd7yknf1YoNbCzqRKc4qyor8gaKU1qmn+CSxhbuBusANI9QpRohGBreCFkKxLhei6S9CQXFEbbKuqLg0DA=="
12    crossorigin="anonymous"
13    referrerpolicy="no-referrer"
14  />
15 </head>
```

Com isso o projeto já está preparado para utilizar os ícones do Font Awesome, para isso vamos realizar uma pesquisa do ícone que desejamos no site:





Construindo um Header Eficiente

Vamos agora substituir os textos dos nossos botões pelo código HTML do ícone. Dessa forma, em vez de textos, teremos ícones exibidos nos nossos botões. E adicionar um margem no elemento **<div>** utilizando a **classe 'mx-2'**.

```
23 <div class="text-slate-200 flex gap-2 items-end mx-2">
24   <button><i class="fa-solid fa-cart-shopping"></i></button>
25   <button><i class="fa-solid fa-user"></i></button>
26 </div>
```



Com isso concluímos a estrutura básica do nosso projeto e vamos implementar o nosso **Carrinho de Compras**.

Parte 5

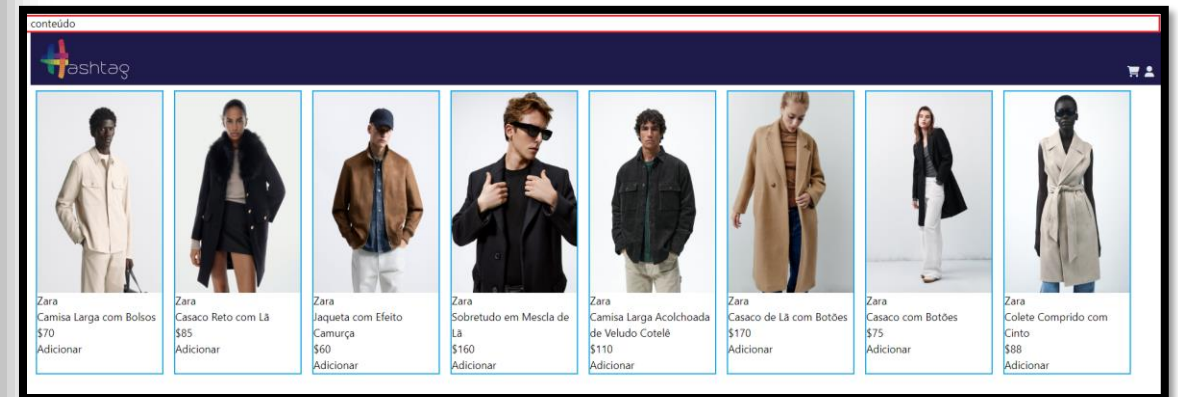
Desenvolvendo o Carrinho de Compras



Desenvolvendo o Carrinho de Compras

Agora que montamos a base do nosso projeto, vamos começar a construir o nosso carrinho de compras. O primeiro passo é criar um **elemento <section>** que vai conter todo o carrinho. Esse elemento **<section>** vai ter um "id". Para entendermos melhor como esse elemento está posicionado, vamos adicionar algum conteúdo dentro dele.

```
16 <body>
17   <section id="carrinho">conteúdo</section>
18   <header class="flex bg-indigo-950 p-2 justify-between">
19     
24     <div class="text-slate-200 flex gap-2 items-end mx-2">
25       <button><i class="fa-solid fa-cart-shopping"></i></button>
26       <button><i class="fa-solid fa-user"></i></button>
27     </div>
28   </header>
29   <main>
30     <section id="container-produto" class="flex flex-wrap"></section>
31   </main>
32   <script src="main.js"></script>
33 </body>
```





Desenvolvendo o Carrinho de Compras

Criado o nosso elemento Carrinho de compras, vamos iniciar os ajustes de posicionamento dele na nossa página.

No contexto do Tailwind CSS, a **classe fixed** é usada para definir a posição de um elemento como fixa. Isso significa que o elemento permanecerá no mesmo lugar, mesmo que a página seja rolada. É semelhante à **propriedade position: fixed**; no CSS regular.

As **classes top-0** e **right-0** são usadas para posicionar o **elemento fixo no topo direito da página**. A classe top-0 define que a borda superior do elemento estará no topo da página e a classe right-0 define que a borda direita do elemento estará alinhada com o lado direito da página.

Adicionaremos um cor de fundo ao nosso elemento, apenas para visualizar o seu novo posicionamento após adição das classes utilitárias do Tailwind.

```
17 <section id="carrinho" class="fixed top-0 right-0 bg-indigo-100">conteúdo</section>
```



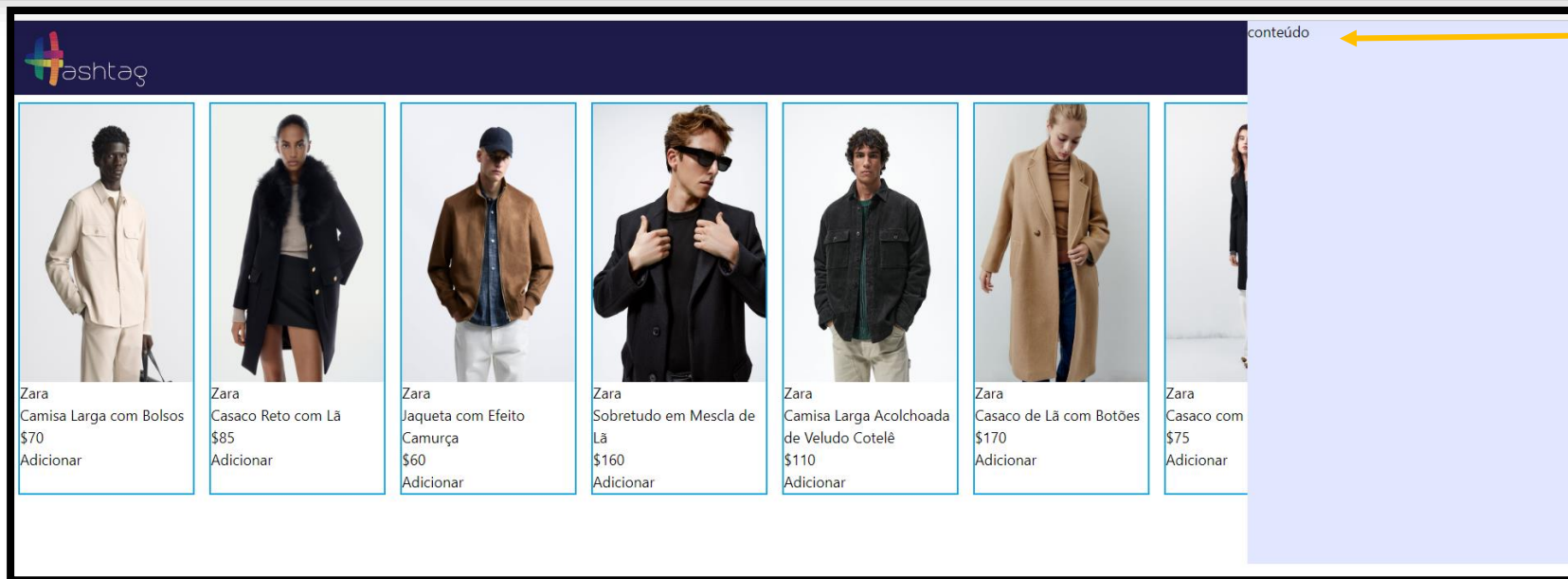


Desenvolvendo o Carrinho de Compras

Vamos configurar o elemento **<section>** para que ele ocupe a altura total da tela, utilizamos a **classe 'h-screen'** que é usada para definir a altura de um elemento igual à altura da tela.

E para a largura do nosso elemento, adicionaremos a **classe w-[360px]** é usada para definir a largura de um elemento para 360 pixels. Isso é parte da funcionalidade de valores arbitrários, ou seja, você consegue adicionar tamanho diferentes dentro **[]** caso o Tailwind não possua a classe desse valor.

```
17 <section id="carrinho" class="fixed top-0 right-0 bg-indigo-100 h-screen w-[360px]">conteúdo</section>
```





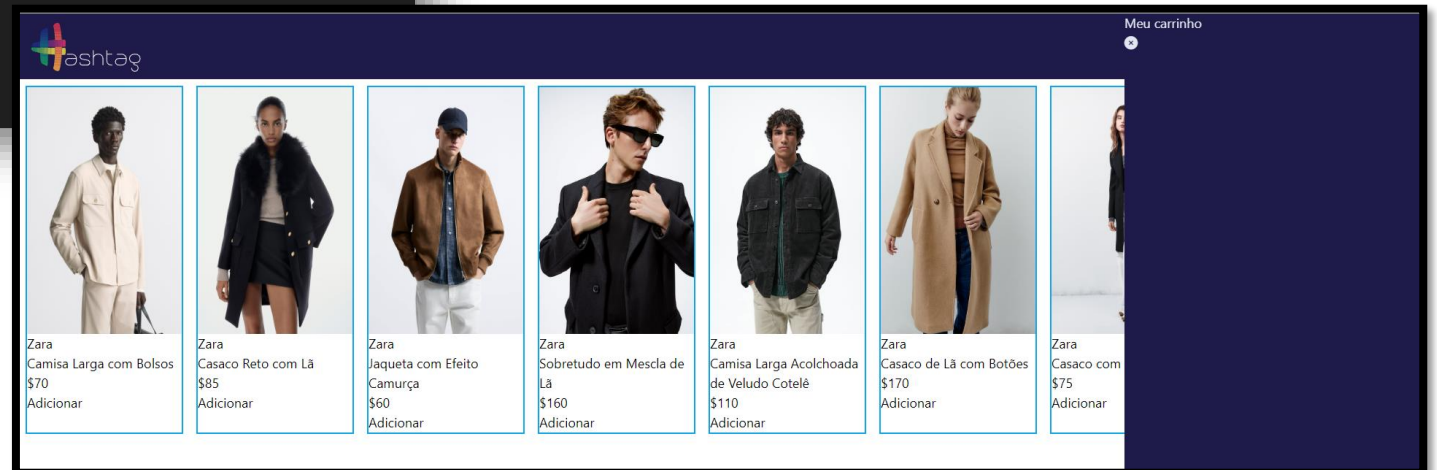
Desenvolvendo o Carrinho de Compras

Agora, vamos mudar a cor de fundo do nosso carrinho de compras para a **classe bg-indigo-950**. Essa é a mesma cor que usamos no nosso Header, porque queremos que o carrinho pareça uma extensão do Header quando ele aparecer no nosso site. Além disso, vamos definir a cor do texto usando a **classe text-slate-200**.

Começaremos a organizar a estrutura do nosso carrinho adicionando um elemento **<div>** com **"id='cabecalho-carrinho'"** que representará o cabeçalho do nosso carrinho de compras.

Dentro do **elemento <div>**, crie um elemento **<p>** que receberá o texto **"Meu carrinho"** e um **botão** para fechar o carrinho, que receberá um ícone do site Font Awesome.

```
17 <section id="carrinho" class="fixed top-0 right-0 bg-indigo-950 h-screen w-[360px] text-slate-200">
18   <div id="cabecalho-carrinho">
19     <p>Meu carrinho</p>
20     <button><i class="fa-solid fa-circle-xmark"></i></button>
21   </div>
22 </section>
```





Desenvolvendo o Carrinho de Compras

Transformaremos o nosso **elemento <div>** que está encapsulando o cabeçalho do nosso carrinho em um container flexível com a **classe flex**.

Além disso, iremos arrumar os **elementos <p> (parágrafo) e <button> (botão)**. Vamos colocar o parágrafo de um lado e o botão do outro, usando a **classe justify-between**. Isso vai deixar eles bem posicionados. E adicionaremos uma margem, com a **classe m-4**.

```
17 <section
18   id="carrinho"
19   class="fixed top-0 right-0 bg-indigo-950 h-screen w-[360px] text-slate-200"
20 >
21   <div id="cabecalho-carrinho" class="flex justify-between m-4">
22     <p>Meu carrinho</p>
23     <button id="fechar-carrinho">
24       <i class="fa-solid fa-circle-xmark"></i>
25     </button>
26   </div>
27 </section>
```





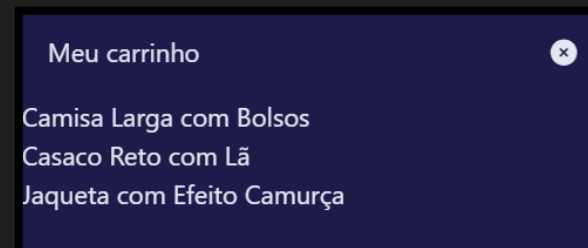
Desenvolvendo o Carrinho de Compras

Continuando a criação dos elementos intermediários do nosso carrinho de compras, adicionaremos mais alguns elementos que serão responsáveis por armazenar as seções dos cards de produtos, uma seção para preço e um botão para finalizar a compra.

Os elementos intermediários terão alguns elementos apenas para visualizarmos as suas posições no carrinho de compras. E ao longo da construção do nosso site vamos modifica-los.

Começando com a seção de cards de produtos:

```
17 <section
18   id="carrinho"
19   class="fixed top-0 right-0 bg-indigo-950 h-screen w-[360px] text-slate-200"
20 >
21   <div id="cabecalho-carrinho" class="flex justify-between m-4">
22     <p>Meu carrinho</p>
23     <button id="fechar-carrinho">
24       <i class="fa-solid fa-circle-xmark"></i>
25     </button>
26   </div>
27   <section id="produtos-carrinho">
28     <p>Camisa Larga com Bolsos</p>
29     <p>Casaco Reto com Lã</p>
30     <p>Jaqueta com Efeito Camurça</p>
31   </section>
32 </section>
```

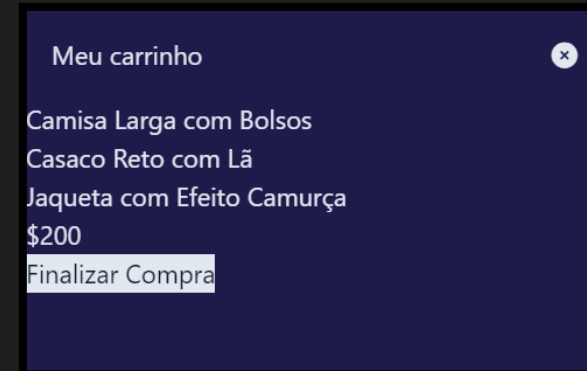




Desenvolvendo o Carrinho de Compras

Adicionaremos o **elemento <p>** para o texto que representará o valor total da compra e um **<button>** para finalizar a compra.

```
17 <section
18   id="carrinho"
19   class="fixed top-0 right-0 bg-indigo-950 h-screen w-[360px] text-slate-200"
20 >
21   <div id="cabecalho-carrinho" class="flex justify-between m-4">
22     <p>Meu carrinho</p>
23     <button id="fechar-carrinho">
24       <i class="fa-solid fa-circle-xmark"></i>
25     </button>
26   </div>
27   <section id="produtos-carrinho">
28     <p>Camisa Larga com Bolsos</p>
29     <p>Casaco Reto com Lã</p>
30     <p>Jaqueta com Efeito Camurça</p>
31   </section>
32   <p id="preco-total">$200</p>
33   <button class="bg-slate-200 text-slate-900">Finalizar Compra</button>
34 </section>
```



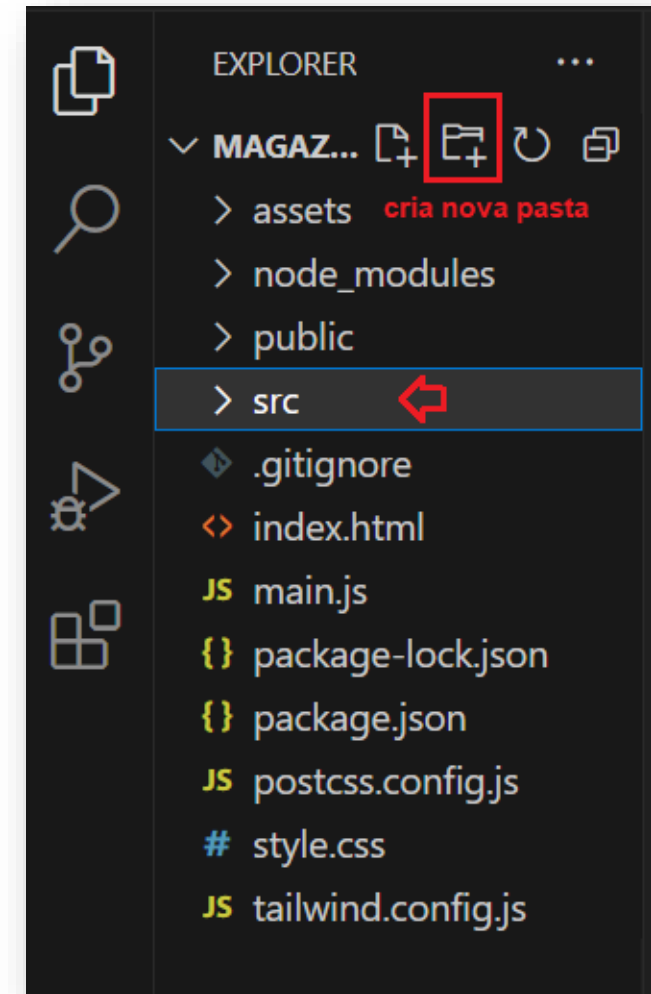


Desenvolvendo o Carrinho de Compras

Neste estágio, abordaremos a implementação da lógica operacional para o nosso carrinho de compras, permitindo que ele seja ativado mediante o clique no ícone correspondente, localizado em nossa barra de cabeçalho ("Header"). Além disso, também será incorporada a funcionalidade para desativar o carrinho, acionada quando o ícone de fechamento (representado pelo ícone "x") for clicado. Essa abordagem visa oferecer ao usuário maior controle sobre a exibição do carrinho, de acordo com suas necessidades.

Vamos criar uma **nova pasta** dentro do nosso projeto chamada **"src"**. Uma pasta "src" é uma convenção comum usada na estruturação de projetos de desenvolvimento de software. "src" é uma abreviação de **"source"** (fonte, em inglês).

A pasta "src" é usada para manter e organizar o código-fonte principal de um projeto. Ela ajuda a melhorar a legibilidade, a manutenção e a escalabilidade do código, seguindo as práticas recomendadas de desenvolvimento de software.

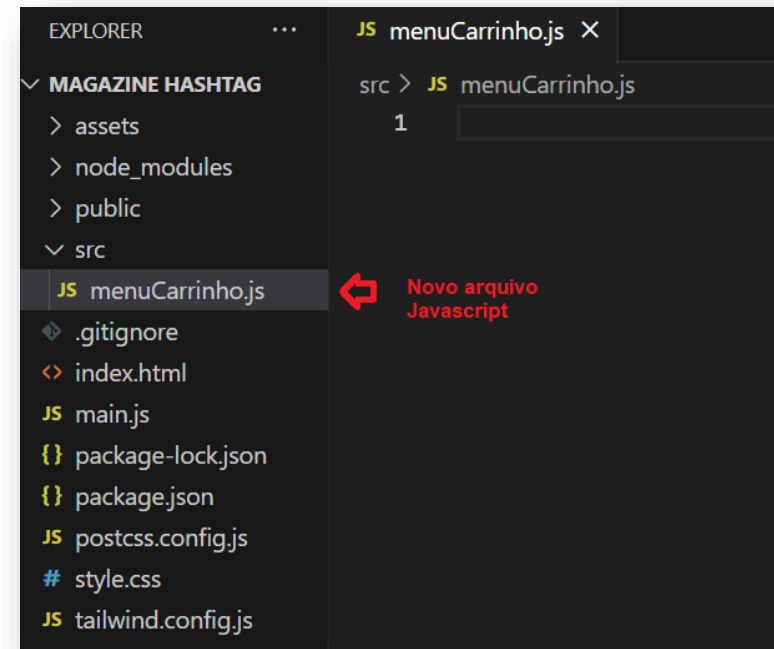
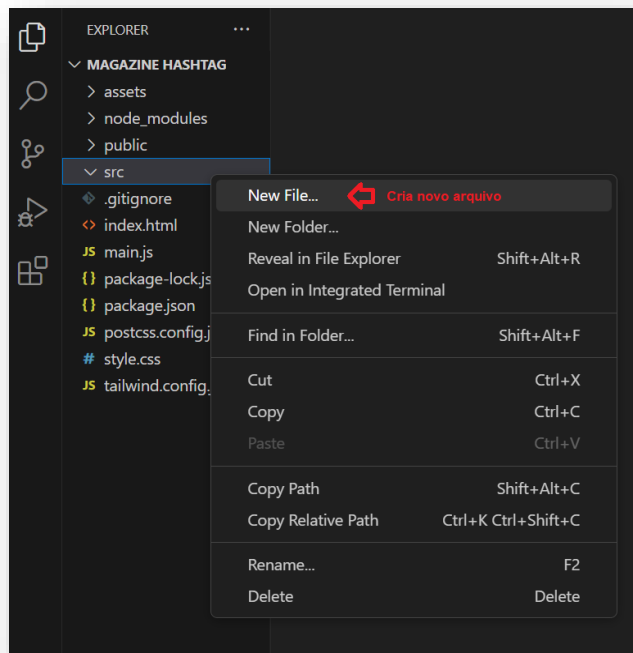




Desenvolvendo o Carrinho de Compras

Dentro da pasta "src" criaremos o nosso segundo arquivo de Javascript, chamado "**menuCarrinho.js**", que será responsável por armazenar a inteligência do carrinho de compras.

Criar vários arquivos JavaScript separados para diferentes funcionalidades do nosso site é uma boa prática porque promove organização, manutenção facilitada, reutilização de código e escalabilidade do projeto. Isso resulta em um código mais limpo, modular e eficiente.



O nosso arquivo main.js é o nosso código principal e por isso ficará na raiz do projeto.



Desenvolvendo o Carrinho de Compras

Para criarmos as ações de abrir e fechar o nosso carrinho de compras precisamos entender sobre funções em Javascript.

Uma **função** em JavaScript é um **conjunto de instruções** que você cria e dá um nome. É como uma "receita" que o computador segue quando você chama essa função pelo nome. Isso ajuda a organizar o código, reutilizar ações e tornar as tarefas mais fáceis de fazer.

Ao nomear funções e variáveis, é recomendado seguir a **convenção camelCase**. Isso significa que você escreve a primeira palavra toda em minúsculas e as palavras seguintes começam com letra maiúscula, sem espaços. Por exemplo, "nomeDaFuncao" segue a convenção camelCase.

Funções podem receber informações (chamadas de **parâmetros**) que funcionam como ingredientes da "receita". Você pode usar esses parâmetros dentro da função para personalizar o comportamento dela. Além disso, as funções também podem retornar resultados, tornando a programação mais eficiente e compreensível.

```
JS menuCarrinho.js ●
src > JS menuCarrinho.js > ...
1  function abrirCarrinho() {};
```



Desenvolvendo o Carrinho de Compras

No caso queremos que o carrinho de compras apareça e desapareça de forma suave, então ajustaremos as classes **"right"** e **"h"** que vão trabalhar juntas para fazer isso acontecer. A **classe "right"** ajuda a posicionar o carrinho inicialmente fora da tela, **360 pixels à esquerda**, escondendo-o (**Imagem 1**). Por outro lado, a **classe "h"** garante que o carrinho, quando estiver visível, ocupe toda a altura da tela.

Combinando essas classes, o carrinho será colocado fora da vista inicialmente. Quando ativado, ele deslizará suavemente para dentro da tela a partir da borda direita, proporcionando a sensação de aparecer, e para isso iremos utilizar a **classe right-[0px]** (**Imagem 2**). E quando desativado, ele deslizará de volta para fora, dando a sensação de desaparecer. Isso cria um efeito agradável de visibilidade e ocultação do carrinho, contribuindo para a experiência do usuário.

1 19 `class="fixed top-0 right-[-360px] bg-indigo-950 h-screen w-[360px] text-slate-200"`

2 19 `class="fixed top-0 right-[0px] bg-indigo-950 h-screen w-[360px] text-slate-200"`



Desenvolvendo o Carrinho de Compras

Nesse momento iremos criar as nossas funções de abrir e fechar o nosso carrinho de compras, e para isso vamos conhecer dois métodos, **classList.remove()** e **classList.add()**. Um **método** é uma ação que podemos chamar em elementos HTML selecionados (como o carrinho) para modificar as classes associadas a eles, controlando assim sua aparência e comportamento.

```
src > JS menuCarrinho.js > ...  
1  function abrirCarrinho() {}  
2  
3  function fecharCarrinho() {  
4      document.getElementById("carrinho").classList.remove('right-[0px]');  
5      document.getElementById("carrinho").classList.add('right-[-360px]');  
6  }
```

Nesse trecho de código, temos uma função chamada **"fecharCarrinho()"**. Essa função é responsável por controlar o comportamento de fechamento do carrinho de compras.

- **document.getElementById("carrinho")**: Estamos usando o método **document.getElementById()** para selecionar o elemento HTML com o ID "carrinho". Isso nos permite interagir com o elemento que representa o carrinho na página.



Desenvolvendo o Carrinho de Compras

- **`classList.remove('right-[0px]')`**: Estamos removendo uma classe chamada 'right-[0px]' do elemento do carrinho. A classe 'right-[0px]' controla a posição do carrinho. Ao remover essa classe, estamos desfazendo essa posição específica.

- **`classList.add('right-[-3600px]')`**: Estamos adicionando uma nova classe chamada 'right-[-3600px]' ao elemento do carrinho. Essa classe define a posição do carrinho para estar fora da tela, a uma distância de 360 pixels à esquerda. Isso fará com que o carrinho "saia" da tela.

Em resumo, a função "**`fecharCarrinho()`**" é usada para controlar o desaparecimento do carrinho de compras da página web. Ela remove uma classe que permite o carrinho aparecer e adiciona uma classe que move o carrinho para fora da tela, dando a impressão de que o carrinho está sendo fechado.



Desenvolvendo o Carrinho de Compras

A função "**abrirCarrinho()**" terá o comportamento oposto a função "fecharCarrinho()", pois queremos que ela remova a classe faz com que o nosso carrinho de compras se desloque para fora da página e adicione a classe que permite o nosso carrinho ser visível dentro da página.

```
src > JS menuCarrinho.js > ...
1  function abrirCarrinho() {
2      document.getElementById("carrinho").classList.add('right-[0px]');
3      document.getElementById("carrinho").classList.remove('right-[-360px]');
4  }
5
6  function fecharCarrinho() {
7      document.getElementById("carrinho").classList.remove('right-[0px]');
8      document.getElementById("carrinho").classList.add('right-[-360px]');
9  }
```

Continuando no processo de desenvolvimento do abrir e fechar o nosso carrinho, precisamos adicionar um "**id**" ao elemento **<button>** que representa o nosso carrinho de compras que está localizado dentro do nosso Header.

```
42  <button id="abrir-carrinho"><i class="fa-solid fa-cart-shopping"></i></button>
```



Desenvolvendo o Carrinho de Compras

Dentro do nosso arquivo `menuCarrinho.js`, vamos criar mais uma função chamada "**inicializarCarrinho()**", que será responsável por preparar a interação do usuário com o carrinho de compras do nosso site.

Essencialmente, a função **inicializarCarrinho()** irá configurar os botões de fechar e abrir o carrinho para responderem a cliques do usuário, executando as funções correspondentes (**fecharCarrinho()** e **abrirCarrinho()**) quando os botões são acionados. Isso permite que o carrinho seja manipulado de acordo com as ações do usuário, proporcionando uma interação fluída e intuitiva.

Para isso, vamos implementar dentro da nossa função o método chamado "**addEventListener**".

O **addEventListener** é como dizer ao JavaScript para ficar "**escutando**" por um evento específico, como um clique em um botão. Quando esse evento acontece, o JavaScript executa uma ação que você especificou, como chamar uma função. É como ter um ouvido atento que reage sempre que algo acontece, como apertar um botão para fazer algo acontecer na sua página da web.



Desenvolvendo o Carrinho de Compras

```
11 function inicializarCarrinho() {  
12     const botaoFecharCarrinho = document.getElementById("fechar-carrinho");  
13     const botaoAbrirCarrinho = document.getElementById("abrir-carrinho");  
14  
15     botaoFecharCarrinho.addEventListener("click", fecharCarrinho);  
16     botaoAbrirCarrinho.addEventListener("click", abrirCarrinho);  
17 }
```

- **const botaoFecharCarrinho = document.getElementById("fechar-carrinho");** - Estamos selecionando o botão que tem o ID "fechar-carrinho" no HTML. Esse botão, quando clicado, servirá para fechar o carrinho de compras.
- **const botaoAbrirCarrinho = document.getElementById("abrir-carrinho");** - Similarmente, estamos selecionando o botão com o ID "abrir-carrinho". Esse botão, quando clicado, abrirá o carrinho de compras.
- **botaoFecharCarrinho.addEventListener("click", fecharCarrinho);** - Aqui, estamos adicionando um "ouvinte" de evento ao botão de fechar carrinho. Isso significa que, quando o botão é clicado, a função **fecharCarrinho()** será chamada para fechar o carrinho.
- **botaoAbrirCarrinho.addEventListener("click", abrirCarrinho);** - De forma semelhante, estamos adicionando um ouvinte de evento ao botão de abrir carrinho. Quando esse botão é clicado, a função **abrirCarrinho()** será chamada para abrir o carrinho.



Desenvolvendo o Carrinho de Compras

Agora que desenvolvemos a lógica para os botões que controlam a abertura e o fechamento do carrinho de compras, precisamos torná-los operacionais em nosso site. Para isso, vamos configurar os botões para funcionarem adequadamente. A fim de realizar isso, vamos exportar a **função "inicializarCarrinho()"** para o nosso arquivo principal de JavaScript, conhecido como main.js. Esse arquivo desempenha um papel central no nosso projeto, onde combinaremos todas as funcionalidades que desenvolvemos.

Para realizar essa exportação, utilizaremos a palavra-chave **"export"**. Essa ferramenta permite compartilhar e reutilizar o código em diferentes partes do projeto. Ao aplicar **"export"** antes de uma função, estamos, de maneira simples, comunicando: "Ei, essa função pode ser usada em outros lugares também!".

```
src > JS menuCarrinho.js > ...
1  function abrirCarrinho() {
2      document.getElementById("carrinho").classList.add('right-[0px]');
3      document.getElementById("carrinho").classList.remove('right-[-360px]');
4  }
5
6  function fecharCarrinho() {
7      document.getElementById("carrinho").classList.remove('right-[0px]');
8      document.getElementById("carrinho").classList.add('right-[-360px]');
9  }
10
11  export function inicializarCarrinho() {
12      const botaoFecharCarrinho = document.getElementById("fechar-carrinho");
13      const botaoAbrirCarrinho = document.getElementById("abrir-carrinho");
14
15      botaoFecharCarrinho.addEventListener("click", fecharCarrinho);
16      botaoAbrirCarrinho.addEventListener("click", abrirCarrinho);
17  }
```



Desenvolvendo o Carrinho de Compras

Agora, no **arquivo main.js**, usaremos a palavra-chave **"import"** para trazer essa função para dentro do nosso código. Dessa forma, podemos usar a **função "inicializarCarrinho()"** que criamos em outros módulos do nosso projeto. Isso ajuda a manter nosso código organizado e reutilizável.

```
JS main.js > [🔗] catalogo
1   import { inicializarCarrinho } from "../src/menuCarrinho";
```

Precisamos lidar com mais um detalhe no **arquivo index.html**, dentro da nossa **tag <script>** iremos adicionar um **atributo type="module"**, que irá informar ao navegador que o arquivo "main.js" é um módulo JavaScript, permitindo a organização modular do código e permitirá o uso das palavras-chave **import** e **export** para compartilhar funcionalidades entre diferentes partes do projeto.

```
<> index.html > 📁 html
49   <script type="module" src="main.js"></script>
```



Desenvolvendo o Carrinho de Compras

Para concluir esta etapa, vamos incorporar a execução ou chamada da nossa função **"inicializarCarrinho()"** no final do nosso arquivo main.js. Isso significa que, quando o arquivo principal do JavaScript for carregado, essa função será ativada, configurando os botões e a interação do carrinho de compras conforme planejamos.

```
JS main.js > ...
74     src="./assets/img/${produtoCatalogo.imagem}"
75     alt="Produto 1 do Magazine Hashtag."
76     style="height: 300px"
77   />
78   <p class="marca">${produtoCatalogo.marca}</p>
79   <p>${produtoCatalogo.nome}</p>
80   <p>${produtoCatalogo.preco}</p>
81   <button>Adicionar</button>
82   </div>`;
83   document.getElementById("container-produto").innerHTML += cartaoProduto;
84 }
85
86 inicializarCarrinho(); Chamada da função
```

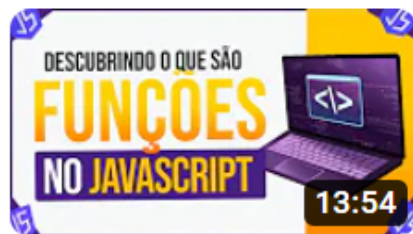
Com essa etapa concluída, conseguimos finalizar os botões que controlam a abertura e o fechamento do nosso carrinho de compras. Isso significa que a funcionalidade que criamos para esses botões está agora em pleno funcionamento, e o carrinho de compras do nosso site está totalmente operacional.

Parte Bônus

Vídeos

Complementares

Vídeos Complementares



O que são Funções no JavaScript?

Hashtag Programação • 6,8 mil visualizações • há 1 mês

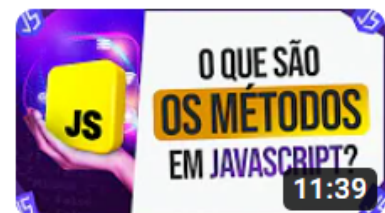
<https://www.youtube.com/watch?v=7GkVh6WP2EQ&list=PLpdAy0tYrnKyLd8e6e3suYCYO3LKzCVzi&index=20>



Parâmetros, Argumentos e Retornos em Funções no Javascript

Hashtag Programação • 936 visualizações • há 1 mês

<https://www.youtube.com/watch?v=Q5-zpqCHav0&list=PLpdAy0tYrnKyLd8e6e3suYCYO3LKzCVzi&index=22>



O que são Métodos em JavaScript?

Hashtag Programação • 878 visualizações • há 13 dias

<https://www.youtube.com/watch?v=LlsLoekKH88&list=PLpdAy0tYrnKyLd8e6e3suYCYO3LKzCVzi&index=25&pp=iAQB>

Ainda não segue a gente no Instagram e nem é inscrito no nosso canal do Youtube? Então corre lá!



@hashtagtreinamentos



youtube.com/hashtag-treinamentos

